

Long-Horizon Practice: Structural Lower Bounds Beyond Single-Step Capability

Sammy Zeng

Dao Wei

2026

¹E-mail: jop.sammy@163.com ORCID: 0009-0007-4382-0040

²This paper is a research outcome of the “Long-Horizon Practice Theory” project, a special-support initiative of the Qianjiang New Town Central Business Industry Community, Shangcheng District, Hangzhou. The community provided critical support in survey coordination, foundational stress-testing samples, and real-world business use cases (non-financial support).

Abstract

Prior discussions of agent capabilities largely default to single-step local closure as the fundamental yardstick, making it difficult to account for the recurrent instability observed in genuinely complex tasks. This paper proposes “Long-Horizon Practice” as a structural concept, defining it as a practice type characterized by a non-precomputable chain of sequential dependencies that must be sustained across checkpoints: long-horizon does not mean longer in time, but rather that critical information must be progressively generated as the practice advances. Starting from the logical fork of “why is the key input currently unavailable,” we exhaustively identify four sources—epistemic gap, medium constraint, decision fork, and external drift—and on this basis construct a three-tier structural lower bound consisting of a substrate prerequisite, six logically necessary conditions, and seven engineering compensation conditions. This reveals a three-link chain of non-implication between single-step capability and long-horizon capability: strong single-step capability $\nRightarrow$ single-checkpoint closure $\nRightarrow$ cross-checkpoint sustained progression $\nRightarrow$ high single-round Benchmark scores. Current evaluation frameworks thus exhibit a fundamental misalignment in their capability scale.

Keywords: Long-Horizon Practice; sequential dependency chain; structural lower bound; agent evaluation; human-machine collaboration

Contents

1	Introduction	5
2	Rigorous Definition of Long-Horizon Practice	7
2.1	Practice and Checkpoints	7
2.2	Static Foldability and Key Input	7
2.3	Formal Definition	9
3	Four Sources of Current Unavailability of Information	11
3.1	Exhaustive Derivation: First Ask Whether the Truth Value Has Been Determined	11
3.2	Epistemic Gap: Unexecuted Means Truth Value Undetermined . .	12
3.3	Medium Constraint: Truth Value Determined, But Cannot Enter the Current Working Set	14
3.4	Decision Fork: Branch Uncommitted Means Required Informa- tion Identity Undetermined	15
3.5	External Drift: The Required Truth Value Belongs to a Future State	16
3.6	State Transfer: The Cost of Acquiring Key Information	16
4	Three-Layer Structural Lower Bound of Long-Horizon Practice	18
4.1	Substrate Prerequisite	18
4.1.1	Topological Limits of the Trajectory	19
4.1.2	Physical Limits of the Engine—LLM’s Carrying Baseline	20
4.1.3	Cognitive Limits of Human Drivers—Human Carrying Base- line	22
4.1.4	Irreplaceability of the Substrate Prerequisite	24
4.2	Logical Necessary Conditions: Preventing Structural Fractures of Continuity	24
4.2.1	Condition 1: Goal closure [hard fracture]	25
4.2.2	Condition 2: Value closure [soft fracture]	25
4.2.3	Condition 3: Checkpoint decomposition [hard fracture] . .	26
4.2.4	Condition 4: Ordering constraint [soft fracture]	26
4.2.5	Condition 5: State transfer [hard fracture]	27
4.2.6	Condition 6: Open-state marking [soft fracture]	27
4.3	Engineering Compensation Conditions: Placing the Skeleton into the Real World	29

4.3.1	Compensation 1: Recovery and rollback	29
4.3.2	Compensation 2: Isolation and boundary	29
4.3.3	Compensation 3: Independent evaluation	30
4.3.4	Compensation 4: Instruction following compensation . . .	30
4.3.5	Compensation 5: Capability gap escalation	30
4.3.6	Compensation 6: Proactive machine-to-human explanation	31
4.3.7	Compensation 7: Proactive machine-to-human inquiry and alignment	31
4.4	Fracture Type to Condition System Mapping	31
5	Cross-Domain Generality: Structure, Not Subject Matter, at Work	33
5.1	Structural Variables and Subject-Matter Variables	33
5.2	Fractal Isomorphism: Construction and Execution Share the Same Constraint Equation	34
5.3	A Minimal Cross-Domain Anchor: Architectural Proposal Review	35
5.4	Closing	36
6	When Single-Step Closure Encounters Long-Horizon Structure: Three Types of Structural Failure	37
6.1	Failure I: Compensating with More Context When Information Has Not Yet Been Generated	37
6.2	Failure II: Substituting Memory for State, Self-Report for Verifi- cation	38
6.3	Failure III: Substituting Local Convenience for Global Judgment at Nodes Requiring Value Adjudication	38
6.4	Closing: A Shared Structural Root Cause	39
7	Conclusions and Corollaries	40
7.1	Corollary: The Three-Stage Non-Implication of Capability Di- mensions	40
7.2	Contributions Recovered by This Paper	41
7.3	Limitations and Boundaries	42
7.4	Open Questions	44
A	Controlled Dissection: A Structural Specimen of Long-Horizon Prac- tice	45
A.1	Problem Statement	45
A.2	Specimen Selection: Why This Problem is Suitable for Dissection	47
A.3	Checkpoint Reconstruction: State Snapshots of Cognitive Progress	48
A.4	Fracture Localization: Concrete Manifestations of the Four Sources in a Closed System	51

A.5	Continuity Repair: How the Carrying Structure Emerges Naturally	52
A.6	Why the Bare Model Consistently Fails	54
A.7	Specimen Boundaries: What It Covers and What It Does Not . . .	55
B	Acknowledgments	56

1 Introduction

In contemporary large language model (LLM) and agent engineering, a recurring phenomenon is that many systems perform well on single-instruction responses, short-context reasoning, and localized code modifications, yet exhibit systemic instability once they enter genuine long-horizon tasks such as cross-cycle code-base migration, sustained literature surveys, or multi-week software maintenance. Even if we provisionally assume that the executor possesses near-infinite local reasoning capability, this disparity does not vanish on its own, because the problem lies not primarily in “insufficient intelligence” but in the fact that the task’s critical information is not all given at the starting point—it must be progressively generated across checkpoints. Long-horizon failure is first and foremost a structural problem, not an intellectual one.

This point can be grasped by examining two limiting cases. Translating a 1,000-page novel involves enormous workload, yet it is typically not long-horizon practice, because each page is largely informationally independent and can be decomposed into a large number of parallel single-step tasks. Diagnosing a machine that “fails to turn on after the power button is pressed” may involve modest workload, yet it inherently possesses a long-horizon structure: even if we assume the repairer has infinite intelligence, before actually probing the environment, he cannot know whether the problem lies in a loose power cord, an unpowered outlet strip, an abnormal display link, or a faulty motherboard; and each new observation from a check further determines what to do next. Long-horizon is not about large workload, but about critical information that is currently unavailable.

This reveals a blind spot in existing discourse paradigms. We mostly default to “single-step local closure” as the basic unit for measuring agent capability, so when long-horizon tasks fail, the common reaction is to continue adding context, prompt chains, external retrieval (RAG), or tool access. Yet these patches all assume the same premise: that the critical information required by the task already exists at the initial moment, and all that remains is retrieval, organization, and execution. Definition 2.4 will show that when a task contains an unprecomputable chain of sequential dependencies, the key input K_{i+1} of step $i + 1$ can only be generated or obtained after step i has been executed; the problem is therefore not how to locate answers within a fixed pool of information, but how to sustain continuity across a chain in which actions and new information constantly co-generate each other.

The main contributions of this paper are as follows:

1. **Definitional contribution:** We propose “Long-Horizon Practice” as a structural concept, defining it as a practice type characterized by a non-precomputable chain of sequential dependencies that must be advanced across checkpoints,

and provide the minimal formal definitions for the checkpoint sequence $\mathcal{C}$, the current information set $\mathcal{I}_i$, the key input K_{i+1} , and sequential dependency (§2).

2. **Source contribution:** Starting from “why is the key input currently unavailable,” we identify four mutually exclusive and exhaustive sources—epistemic gap, medium constraint, decision fork, and external drift—and explain why each constitutes a structurally inescapable fork (§3).
3. **Lower-bound contribution:** We propose a three-tier structural lower bound consisting of a substrate prerequisite, six logically necessary conditions, and seven engineering compensation conditions, showing what irreplaceable tier constraints the sustainability of long-horizon practice depends upon (§4).
4. **Generality contribution:** We demonstrate that the above definitions and lower-bound framework do not depend on coding scenarios per se, but apply to all sustained practices that contain chains of sequential dependency (§5).
5. **Explanatory contribution:** Applying the definitional lens of this paper to three recurrent classes of structural failure in current long-horizon systems—context enlargement cannot compensate for ungenerated truth values, memory substitutes for structured state, and local convenience substitutes for global value adjudication—we show that they share the same structural root cause (§6).
6. **Scale contribution:** We prove a three-link chain of non-implication between single-step and long-horizon capability: strong single-step capability $\nRightarrow$ single-checkpoint closure $\nRightarrow$ cross-checkpoint sustained progression $\nRightarrow$ high single-round Benchmark scores, and based on this we point out the capability-scale misalignment in current evaluation frameworks (§7).

This paper uses only minimal concrete anchors in the main text; Appendix A provides a complete controlled case as a canonical “dissection specimen”—using a combinatorial-mathematical problem with mechanistically decidable input-output, strictly defined closure criteria, and fully traceable exploration trajectories as the specimen, it shows how structural fractures in long-horizon practice and their continuity repairs manifest on real task trajectories. The complete raw research materials of this specimen (including all checkpoint execution trajectories, falsification histories, anchor files, etc.) have been open-sourced at https://github.com/jopsammy/task1_raw_repo.

2 Rigorous Definition of Long-Horizon Practice

This paper is not concerned with an abstract notion of “tasks being long,” but with a class of sustained practices possessing a specific structural signature: the executor advances a task around some local purpose (telos), yet the key information needed for the next step cannot always be assembled at once within the current working set. To avoid misinterpreting “long-horizon” as “more steps” or “larger workload,” the minimal ontological premises are first established, then decidable symbolic objects are used to demarcate when a process genuinely enters the long-horizon practice regime.

2.1 Practice and Checkpoints

Practice in this paper does not generically denote any execution behavior, but refers to a process in which an executor (human, AI, or a human-machine composite) continuously undergoes the closed loop of “perception $\rightarrow$ judgment $\rightarrow$ action $\rightarrow$ saving intermediate state” around a local purpose. So long as this closed loop continues, the system is not spitting out an answer in one shot, but continually revising subsequent steps through collisions with the environment.

Hence, “long-horizon” must be followed by “practice.” The former only describes the manifest span; the latter identifies the structural essence: subsequent paths are not all given at the starting point, but are progressively determined as the process advances. If a task is long and large, but all critical premises are already stably present at the starting point, then it is merely a large-scale single-step closure; only when the key input of a subsequent step cannot be folded by the current local closed loop does it enter the long-horizon practice discussed in this paper.

Definition 2.1 (Practice instance and checkpoints). A practice instance is denoted as

$$\Pi = (\mathcal{A}, g, \mathcal{C}, \{\mathcal{I}_i\}_{i=0}^n),$$

where $\mathcal{A}$ denotes the executor, g denotes the current local purpose, and $\mathcal{C} = (c_0, c_1, \dots, c_n)$ denotes a finite sequence of checkpoints. A checkpoint c_i is not a single atomic action, but a position where one local closed loop temporarily concludes and the system can stably rest and continue advancing; $\mathcal{I}_i$ denotes the current information set that can be stably invoked at time c_i .

2.2 Static Foldability and Key Input

The purpose of defining checkpoints is not to number steps, but to determine whether the materials needed for the next step are already contained within the

current information set. The following definition provides a precise form of this criterion.

Definition 2.2 (Static foldability). For any checkpoint c_i , if an information object X can be obtained through finite internal computation relying solely on the materials already possessed in $\mathcal{I}_i$, without altering the state of the external world, without executing new probing actions, and without committing to any yet-unselected branch, then X is said to be **statically foldable** with respect to $\mathcal{I}_i$, denoted

$$X \in \text{Fold}(\mathcal{I}_i).$$

Definition 2.2 deliberately excludes three kinds of things often conflated with “currently known”: first, feedback that can only be generated after an experiment or probe has been performed; second, information whose type is unknown until a particular branch has been selected; and third, truths that only become valid after the external world has further evolved. None of these can be counted as inputs directly foldable within the current local closed loop.

With the above object, two classes of processes can be distinguished more precisely. If the key information required by every subsequent step consistently falls within the static folding scope of the current working set, the system—however large the workload—still constitutes an extended execution of single-step closure; if the key input after a certain step cannot be statically folded at the current checkpoint, the system must advance across checkpoints.

Definition 2.3 (Key input and sequential dependency). For the transition from c_i to c_{i+1} , if there exists a minimal information object K_{i+1} such that the actions of executing or verifying c_{i+1} must take it as a premise, then K_{i+1} is called the **key input** of this transition. If

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

then c_{i+1} is said to have a **sequential dependency** on c_i , denoted $c_i \mapsto c_{i+1}$.

The point of Definition 2.3 is not the commonplace observation that “later steps depend on earlier ones,” but that it tightens the dependency criterion to: whether the minimal key input needed by the later step can be statically folded at the current checkpoint. Only when the answer is negative is the earlier step not a mere preamble, but a genuine information-generating or information-shaping event.

Two extreme examples are used below to draw the boundary. They deliberately disentangle workload from structural signature.

Example 2.1 (Translating an extremely long novel). Translating an extremely long novel involves immense workload, yet it typically does not constitute long-horizon

practice. The reason is not that the task is small, but that the source material needed for translating each page is given at the start; the key input of page n does not depend on new feedback that only becomes available after page $n - 1$ has been executed. For page-level subtasks, the key input can, at the initial moment, fall within the static folding scope of some current information set; hence this is massive parallel execution rather than a non-precomputable chain of sequential dependencies.

Example 2.2 (Diagnosing “the computer fails to light up after the power button is pressed”). Diagnosing the fault “the computer fails to light up after the power button is pressed” involves very modest workload, yet it necessarily constitutes long-horizon practice. Whether the first step concerns the power supply can only be decided by the feedback produced after an actual check; whether the second step inspects the display, the host, or the motherboard must likewise be determined by the state newly generated from the preceding step. Even assuming an infinitely intelligent executor, he cannot predict this feedback before colliding with the environment, because this feedback is not a pre-existing answer hidden somewhere awaiting retrieval, but a result produced by the probing action itself.

Example 2.1 and Example 2.2 jointly illustrate: the core of long-horizon practice lies not in the number of steps, duration, or computational demand, but in the presence of key inputs that cannot be statically folded. Once such input appears, the path is no longer “known at the start, remaining only for execution,” but becomes “execution itself alters the information condition needed by the next step.”

(Appendix A provides a more systematic contrast specimen: in a purely formal system, the closure criteria are entirely mechanistically decidable, yet the key input is still progressively generated along checkpoints—long-horizon-ness here does not arise from external noise, but from the internal information structure of the task.)

2.3 Formal Definition

Based on the above, long-horizon practice can be written as a minimal decidability sentence.

Definition 2.4 (Long-horizon practice interval and long-horizon practice task). If for some checkpoint transition $c_i \rightarrow c_{i+1}$, the key input K_{i+1} satisfies

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

then the transition is said to enter a **long-horizon practice interval**. If the execution path of a practice instance Π contains at least one long-horizon practice interval, then the instance is called a **long-horizon practice**.

Definition 2.4 directly provides the boundary between single-step complex tasks and long-horizon practice. Single-step complex tasks can be long, heavy, and difficult, but so long as their key input is consistently statically foldable from the current information set, they remain within the engineering scaling of single-step closure; long-horizon practice is different—after a certain step, the key input lies outside the folding closure of the current working set, so the system must advance across checkpoints. The constraint here is not “the model is not yet smart enough,” but “the currently available set of information is not closed with respect to the next step.”

It is important to emphasize that this paper does not claim all tasks are long-horizon practice. On the contrary, the value of Definition 2.4 lies precisely in tightening the boundary: only when genuinely non-statically-foldable key input actually appears are the structural lower bounds of subsequent sections triggered. If a task remains consistently within the single-step closure regime, then conventional single-step capability suffices, without the need to introduce additional long-horizon support mechanisms.

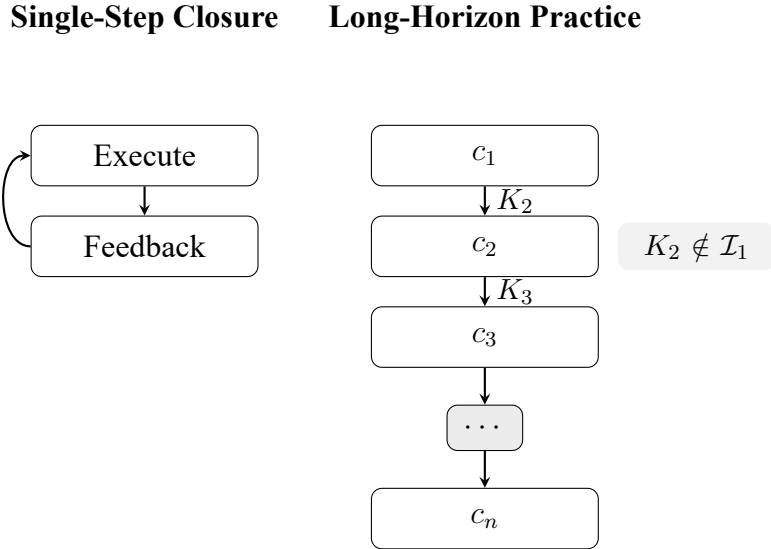


Figure 1: Single-step closure vs. long-horizon practice. Left: all information is available at the start; a single closed loop suffices. Right: the key input K_{i+1} of a later checkpoint is unavailable in the current information set $\mathcal{I}_i$; progress must span checkpoints.

3 Four Sources of Current Unavailability of Information

Definition 2.4 writes long-horizon practice as a decidable condition: at some checkpoint c_i , the key input K_{i+1} needed for the next step does not belong to $\text{Fold}(\mathcal{I}_i)$. But this definition does not yet answer another question: why would a key input fall outside the current folding closure? The core approach of this section is to interrogate a more fundamental condition: **whether the truth value that the next step truly needs to be satisfied has already been stably determined at the current checkpoint.**

3.1 Exhaustive Derivation: First Ask Whether the Truth Value Has Been Determined

For the advancement from c_i to c_{i+1} , merely saying “a certain key input K_{i+1} is needed” is still insufficient, because the same truth value can have different concrete carriers. To this end, we first define the object that the next step genuinely needs to be fixed.

Definition 3.1 (Next-step truth value requirement). For any checkpoint transition $c_i \rightarrow c_{i+1}$, let R_{i+1} denote the minimal truth value that must be fixed before c_{i+1} can be executed or verified. Any concrete data, record, feedback, or environmental reading that can stably witness R_{i+1} counts as a realization of the key input K_{i+1} for this transition.

The role of Definition 3.1 is to separate “what information is needed” from “what carrier bears this information.” Consequently, the reason for $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ can only originate from two root nodes:

1. **The truth value has not yet been stably determined:** at time c_i , R_{i+1} itself has not been fixed.
2. **The truth value has been determined, but cannot be stably imported under current conditions:** R_{i+1} has been fixed, but its witness cannot stably enter $\mathcal{I}_i$ under current media conditions.

The first category further forks according to “what determines the final form of R_{i+1} ,” yielding three mutually distinct mechanisms: the feedback of the current action determines it—this is the epistemic gap; the yet-unselected branch determines it—this is the decision fork; the subsequent evolution of the external world determines it—this is external drift. The second category corresponds to medium constraint. This yields the following proposition.

Proposition 3.1 (Four sources of current unavailability of information). *Under the principle of minimal primary-cause classification, any instance of $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ must fall into one of the following four categories:*

1. **Epistemic gap:** R_{i+1} is determined by the feedback produced after the current step is executed; until execution, the truth value is undetermined.
2. **Decision fork:** R_{i+1} is determined by a yet-uncommitted intermediate branch; until the branch is selected, the identity of the required information is undetermined.
3. **External drift:** R_{i+1} depends on the actual state of the external world at a subsequent moment, and is therefore not yet stably collapsed at time c_i .
4. **Medium constraint:** R_{i+1} has been determined, but its witness cannot be stably imported into $\mathcal{I}_i$ under current medium, bandwidth, attention, or window budget.

Proposition 3.1 explicitly places external drift on the “truth value not yet stable” side, rather than the “already exists but unreachable” side. If the true requirement of the next step is the real-world state at t_{i+1} , then a snapshot from t_i , however accurate and accessible, is not the required truth value itself—it is merely a historical sample that will expire. The drift problem is therefore first and foremost about truth value undetermined, not about limited access.

To avoid superficial overlap in compound tasks, this paper adopts minimal primary-cause classification: if multiple mechanisms are simultaneously present, the classification follows the earliest layer that prevents R_{i+1} from being fixed or imported. After this treatment, the four categories are mutually exclusive in analytical scope and exhaustive in mechanism space.

3.2 Epistemic Gap: Unexecuted Means Truth Value Undetermined

The epistemic gap corresponds to the case where R_{i+1} must be decided by the result produced after the current action collides with the environment. Formally,

$$R_{i+1} = \phi(Y_i),$$

where Y_i is a result that is only generated through probing, experimentation, execution, or interaction when advancing from c_i ; before the action occurs, Y_i does not belong to the current information set, and hence R_{i+1} cannot be fixed either.

The “diagnosing a computer that does not light up” example in Definition 2.4 is precisely such a case: whether power is flowing, whether the display lights up,

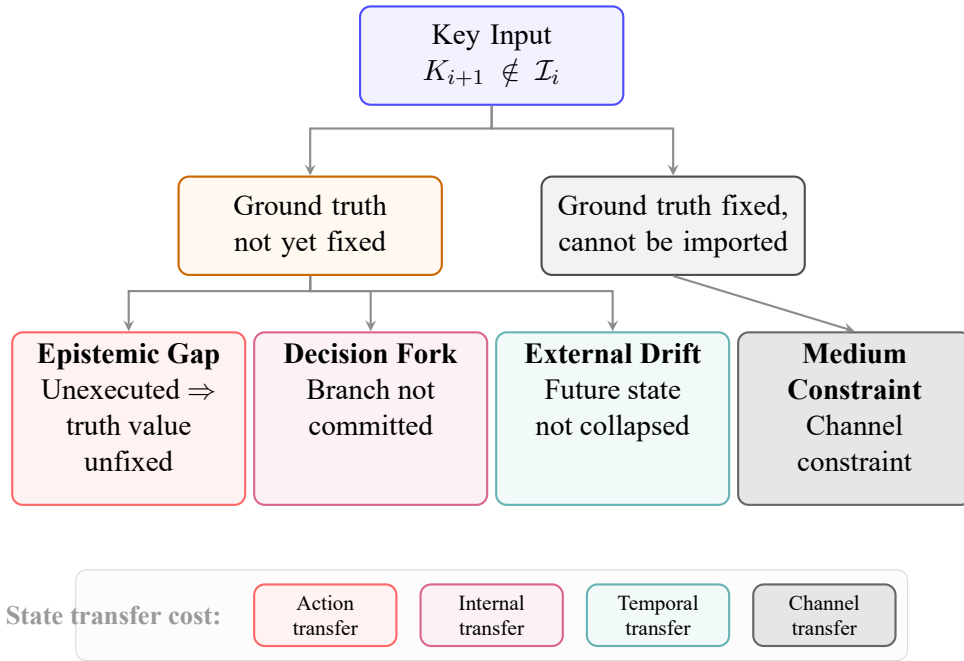


Figure 2: The four exhaustive sources of currently unavailable information. First-level fork at the root node: has the truth value been stably determined? The branch “truth value not yet fixed” further forks by “what determines the final form of R_{i+1} ” into epistemic gap, decision fork, and external drift; the “truth value fixed” branch maps to medium constraint.

and whether the host performs self-test are not answers already hidden in some working set before the check; they are the results of the checking action itself. Unexecuted means unknown outcome, and this “unknown” is not subjective ignorance but the objective absence of the corresponding truth value having been formed.

In the controlled specimen of Appendix A, this mechanism appears repeatedly: whether a candidate route is viable—its truth value is not given at the starting point—it can only be fixed after the exhaustive verification script has run and been compared against the actual data. Unexecuted means truth value undetermined: this is the pure form of the epistemic gap.

The crux of the epistemic gap is that unpredictability does not arise from the practitioner’s insufficient intelligence, but from the generative timing of the information itself. So long as the truth value has not yet been produced by the world, no executor, however strong, can fold it in advance into the current working set.

3.3 Medium Constraint: Truth Value Determined, But Cannot Enter the Current Working Set

Medium constraint describes a different situation: R_{i+1} has been fixed, and its witness truly exists in the world, but this witness cannot be stably loaded into $\mathcal{I}_i$ under current medium conditions. Formally, there exists some witness K_{i+1}^* such that $K_{i+1}^* \models R_{i+1}$, but under the current transport operator τ_i ,

$$\tau_i(K_{i+1}^*) \not\subseteq \mathcal{I}_i.$$

The failure here may arise from bandwidth, window size, noise, attention decay, retrieval cost, or carrier-switching cost.

The medium constraint is by no means unique to LLMs. Human working memory is likewise bounded by attentional ceilings and memory capacity; databases, notebooks, and email archives may all preserve the correct clue, but “being saved” does not equal “having entered the current working set.” The moment retrieval, loading, and re-comprehension themselves require additional steps, medium constraint has already appeared. A more foundational treatment of this constraint comes from Liu et al.^[1], whose agent-level analysis shows that an Omega-agent—an agent capable of accessing all existing information with zero cost, zero delay, and zero loss—cannot exist at the level of physical reality. Medium constraints are objective physical limits, not transient shortcomings of a particular technology.

The context-window limit of LLMs is the most direct instance of this mechanism. Controlled experiments by Liu et al.^[2] on multi-document question answering and key-value retrieval tasks have confirmed the systematic nature of this

“exists but unavailable” phenomenon: even for explicitly long-context models, the retrieval accuracy for critical information located in the middle of the context drops from over 70% at the beginning and end to below 50%, and this U-shaped decay intensifies as the window grows—window size is not equivalent to information accessibility. A certain definition is plainly written at the very front of the input, yet cannot be stably invoked in a long context; the information objectively exists, but cannot enter the current decision position with sufficient fidelity.

Thus, the medium constraint is not “information does not exist,” but “truth value is fixed, yet access is limited.” This differs from epistemic gap, decision fork, and external drift: the latter three involve problems occurring before the truth value has been fixed, whereas medium constraint involves problems occurring after the truth value has been fixed.

3.4 Decision Fork: Branch Uncommitted Means Required Information Identity Undetermined

The distinctiveness of the decision fork lies in this: the problem is not that some known piece of information cannot be obtained, but that it is not yet known which type of information will be needed next. Let B_i be an intermediate branch variable at c_i that has not yet been committed, and

$$R_{i+1} = R_{i+1}^{(B_i)}.$$

So long as B_i remains undetermined, the truth condition that needs to be fixed next does not itself have a unique identity, and therefore no concrete K_{i+1} can be fully specified at the current checkpoint.

For example, when the decision between “rewrite the module” and “patch along the old architecture” has not yet been made, the risks to verify, the data to supplement, and the dependencies to scan after choosing A are entirely different from those after choosing B. Similarly, during debugging, if the logs already contain clues then path A is taken, whereas if the logs are blank then path B is taken; before checking the logs, the executor does not even know for which path to gather subsequent materials.

The decision fork is also not a matter of “not smart enough to think through everything in advance.” Even if a system could enumerate all candidate branches and pre-collect and verify subsequent information for every branch, the total cost would rapidly explode with the number of consecutive fork points— k consecutive binary decision points produce 2^k candidate paths, the vast majority of which will never actually be executed. The difficulty of forking lies not in the enumeration itself, but in the fact that unselected paths also demand real costs of information acquisition and verification.

In the records of Appendix A, the consecutive forks and prohibitions along routes A through L are a direct manifestation of this mechanism: each excluded route consumed real exploration and verification cost, and the exclusion conclusion itself must be stably preserved as a permanent boundary for subsequent search space.

3.5 External Drift: The Required Truth Value Belongs to a Future State

The key to external drift is not that “old information went bad,” but that the truth the next step truly needs corresponds to the external state at a future moment. Let $W(t)$ denote the state of the external world. If

$$R_{i+1} = \psi(W(t_{i+1})),$$

and $W(t)$ continues to evolve independently of the executor over the interval $[t_i, t_{i+1}]$, then even if $\psi(W(t_i))$ is fully available at present, it cannot substitute for $\psi(W(t_{i+1}))$. What is missing here is not a retrieval action, but the fact that the future real state has not yet stabilized.

This is why third-party library version changes, open web content updates, collaborative documents being modified by others, market price fluctuations, and newly added papers in literature databases all drag the system back into a long-horizon chain. The information you hold at time c_i may be correct, but it is not necessarily still correct for c_{i+1} , because what the next step truly needs is a truth that still holds at that time, not a correct snapshot from some past moment.

Hence, external drift and medium constraint must be strictly distinguished. Under medium constraint, the required truth has been fixed, only the carrier cannot come in; under external drift, the required truth itself has not yet stabilized at the current moment. The former is an access problem; the latter is a temporal problem.

3.6 State Transfer: The Cost of Acquiring Key Information

The four sources above appear to belong to distinct causal mechanisms, but they share a deeper structural fact. Recall the criterion of Definition 2.2: can a given information object be obtained through internal computation based solely on existing materials? This definition essentially draws a boundary between “static computation” and “state transfer.” Inside the boundary, information can be obtained through feedforward computation; outside the boundary, information can only be generated or imported at the cost of a state transfer.

The four sources of current unavailability of information correspond precisely to four kinds of state transfer that must occur:

Source Type	Required State Transfer		Essence
Epistemic gap	Action state transfer		Execute probing, collide with the environment to generate feedback; only then is the truth value generated
Decision fork	Internal state transfer		Abandon other branches, collapse to a unique path; only then is the required information identity established
External drift	Temporal state transfer		Wait for the world to evolve to the corresponding moment; only then does the truth stably collapse
Medium constraint	Physical/channel transfer	state	Expend energy or action to transport the already-fixed truth value from cold storage into the current working set

The irreplaceability of these four kinds of state transfer arises from the same root cause: the underlying assumption of the single-step closure paradigm is that “information is given and only needs computational power to be mined.” The contrary fact revealed by long-horizon practice is that information is not an a priori given. It is a byproduct of state transfer. If the corresponding type of state transfer does not occur, the information logically does not exist or is undefined in the current context, rather than merely “exists but cannot be reached.”

This is why strong single-step capability cannot bridge the gap of long-horizon practice: feedforward computation cannot substitute for state transfer. This is not a matter of capability, but a category error—one attempts to replace a necessary causal event with a static operator.

This conclusion also explains why long-horizon practice invariably imposes additional demands on the substrate system. Since state transfer is irreplaceable, a system cannot sustain full-course progression by relying solely on one-shot inferential capability, but must architecturally provide engineering support for these four kinds of state transfer: action state transfer demands continuous probing and verification loops; internal state transfer demands explicit branch commitment and boundary preservation; temporal state transfer demands resampling of external states and temporal validity judgments; channel state transfer demands cross-medium information transport and fidelity verification. These demands are not the icing on the cake of “best practices,” but the structural skeleton that a task

is forced to grow when information is not closed—they form the logical starting point of the three-tier structural lower bound to be unfolded in the next chapter.

4 Three-Layer Structural Lower Bound of Long-Horizon Practice

Definition 2.4 defines the key characteristic of Long-Horizon Practice—key inputs between checkpoints are not given at the starting point; §3 proves that such fractures arise from four structural sources—epistemic gaps, medium constraints, decision forks, and external drift—and are not incidental. Since fracture zones cannot be eliminated, the question becomes: what minimum conditions a system must satisfy to sustain forward progress across these fracture zones?

The answer in this section is not a flat list but a rigorously stratified three-layer system. The first layer, the substrate prerequisite, answers the admission boundary—if unmet, the system should not be used, and no engineering patch can compensate it into a viable carrier system. The second layer, six logical necessary conditions, answers the structural skeleton—once admission is established, how continuity is prevented from being logically severed. The third layer, seven engineering compensation conditions, answers the noise buffer—in a real human-machine environment, how to reduce high-probability failures. The three layers are not interchangeable: the compensation layer can only reinforce the skeleton, not replace it; the logical layer can only define continuity, not bypass the admission boundary.

4.1 Substrate Prerequisite

The substrate prerequisite does not discuss “doing well enough”; it only discusses “whether one has admission qualification.” A system may perform extremely well on single-step benchmarks yet still lack long-horizon qualification—benchmarks measure tasks where information is complete, the path is known, and single-step closure is possible; Long-Horizon Practice demands continuity under conditions where information is not yet complete, the path must be discovered step by step, and execution will be repeatedly interrupted. These are not the same capability dimension.

The carrier is not an abstract “model” but a joint cognitive system composed of humans, LLMs, and tool substrates. The substrate prerequisite unfolds in a three-level deductive manner: first define the topological requirements of the long-horizon practice trajectory—which any executor must satisfy; then derive from trajectory requirements the physical limits that the LLM must satisfy; finally de-

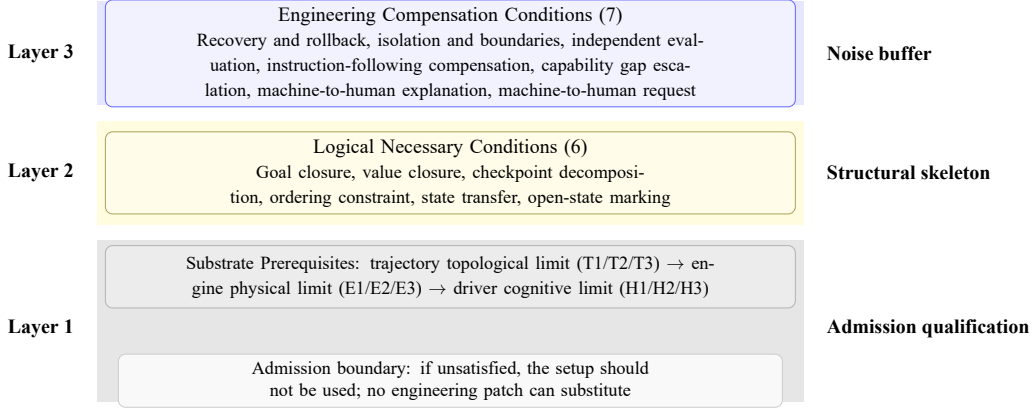


Figure 3: The three-layer structural lower bound of long-horizon practice. Layer 1, substrate prerequisites, defines carrying capacity (the three-level deduction: trajectory topology → engine physics → driver cognition). Layer 2, logical necessary conditions, provides the structural skeleton across checkpoints. Layer 3, engineering compensation conditions, reduces high-probability failures amid real-world noise. The layers are not mutually substitutable.

rive from trajectory and engine constraints the cognitive limits that the human must satisfy. All three levels are deduced in sequence; none can be omitted.

4.1.1 Topological Limits of the Trajectory

The orbital topological limit answers one question: what must a continuous long-horizon practice chain satisfy structurally? These requirements are independent of who executes—a purely human team, a purely LLM system, or a human-machine composite must all satisfy them.

T1: State-snapshot reentrancy. The system must be able to restart from a state-snapshot after an interruption—a persisted context snapshot—and correctly continue execution, rather than starting from scratch or from an erroneous state. Long-Horizon Practice inevitably experiences interruptions: medium constraints clear the context window, and human attention dissipation takes the human offline. If after every interruption one can only restart from zero, Long-Horizon Practice degenerates into a single-step task that “must finish in one stretch or collapse entirely”—this is not long horizon, this is a temporally stretched single step. When T1 is absent, the practice cannot be physically resumed, constituting a hard fracture.

T2: In-context self-verifiability. At any checkpoint, the system must be able to verify, within the currently available context—without introducing external information beyond the current information set—whether the output sat-

isfies preset correctness criteria. Let the verification function at checkpoint c_i be $V_i : (O_i, I_i) \mapsto \{\text{pass}, \text{fail}\}$; T2 requires that V_i 's judgment depends only on I_i : any two verification scenarios with identical context I_i must yield the same judgment from V_i . If verifying the correctness of c_i requires reliance on future information (“we’ll come back and check after finishing later”), then the closure judgment of c_i is unreliable at the moment it occurs—pseudo-closures propagate along the dependency chain and cannot be detected on the spot, constituting a soft fracture.

T3: Causal chain transparency. No knowledge black hole may form between checkpoints. A subsequent executor must be able to reconstruct the causal chain of “why it went this way from the previous checkpoint to here.” If the causal chain is severed, the subsequent executor receives an isolated result—not knowing what assumptions it is based on, what alternatives have been excluded, or what known risks exist. “Continuation” then becomes blind appending rather than judgment-informed advancement. When T3 is absent, subsequent checkpoint judgments lack an informational basis, and the direction may drift, constituting a soft fracture.

These three trajectory requirements form an execution loop: T1 ensures that one can reconnect, T2 ensures that after reconnecting one can self-verify, and T3 ensures that the person reconnecting understands the antecedents and consequences. If any of the three is missing, the trajectory itself lacks the physical structure to carry long-horizon practice.

4.1.2 Physical Limits of the Engine—LLM’s Carrying Baseline

Trajectory requirements are not empty talk—they directly impose physical constraints on the LLM that performs the computation. Here, “physical” does not refer to hardware but to the structurally insurmountable capability boundaries of the LLM as an information processing node.

E1: Window performance span $\geq$ minimum verifiable step length.

This is the most central theoretical contribution within the substrate prerequisite. First, introduce three quantities:

Definition 4.1 (Checkpoint decomposition). Decompose a long-horizon task into a checkpoint sequence $\{c_1, \dots, c_n\}$, where each c_i contains the input context I_{i-1} , output artifact O_i , and closure criterion R_i .

Definition 4.2 (Minimum verifiable step length $L_{\min}$). $L_{\min}$ is the minimum effective output quantity corresponding to the smallest semantic unit that can be independently self-verified (T2). If the output granularity of c_i is smaller than $L_{\min}$, then no independent and effective judgment criterion can be written for R_i —“whether one line of code is correct” has no local semantics; it only has meaning

when placed within a larger semantic unit. $L_{\min}$ is not a constant; it is determined by the minimum verifiable semantic granularity of the task domain.

Definition 4.3 (LLM effective execution span S). S is the maximum effective output quantity that the LLM can stably produce, verifiable by rubric, within a single context window. S is not the context window length but the capability span within which the LLM can stably produce artifacts that pass R_i .

Now consider the tension between the two. For each checkpoint c_i , its output quantity $|O_i|$ is simultaneously squeezed from two directions: downward, $|O_i| \geq L_{\min}$ —otherwise T2 fails and the checkpoint cannot be independently verified; upward, $|O_i| \leq S$ —otherwise the LLM cannot stably complete the checkpoint within a single window.

Theorem 4.1 (E1 infeasibility theorem). *If $S < L_{\min}$, then there exists no checkpoint sequence $\mathcal{C} = (c_1, \dots, c_n)$ such that the output quantity $|O_i|$ of each checkpoint simultaneously satisfies:*

$$\forall c_i \in \mathcal{C} : \quad L_{\min} \leq |O_i| \leq S.$$

This theorem reveals a binary dilemma: when the LLM’s single-step performance S is too weak to cover one independently verifiable minimum semantic unit $L_{\min}$, the system can only choose one of two dead-end paths—either slice checkpoints narrow enough (satisfying $|O_i| \leq S$), but the sliced checkpoints become unverifiable independently (T2 fails, pseudo-closures propagate); or maintain the verification granularity of checkpoints ($|O_i| \geq L_{\min}$), but the span exceeds the LLM’s capability, and the checkpoint collapses internally. Both paths lead to long-horizon bankruptcy.

This is precisely the dimension that benchmarks cannot measure. Benchmark problems have boundaries fixed before starting, correctness criteria known, and required information already complete—they measure the capability of “solving problems with well-defined boundaries.” E1, in contrast, measures “whether the span of each step in an information-incomplete task is sufficient to cover one verifiable semantic unit.” These are not the same dimension—a model excelling in single-turn benchmarks may still gradually drift away from the task’s original specifications and constraints during sustained execution. This accumulated drift, a form of failure mode, is fundamentally unmeasurable by any single-turn evaluation paradigm.

E2: No information loss. When crossing checkpoints, the LLM must not cause information loss due to internal knowledge gaps or context truncation—including both “what happened before” (external contextual information) and “how to do things in this domain” (internal knowledge coverage). T3 requires that causal

chains be reconstructible; if the LLM loses the previous key context between checkpoints, subsequent outputs become rootless—the causal chain fractures into a black hole. T1 requires restarting from a state-snapshot; if the LLM’s internal knowledge after restart does not recall certain domain conventions or norms, execution after reentry may deviate from the original path. Knowledge gaps cannot be resolved by adding more checkpoints or improving contracts—these means themselves depend on the LLM being able to correctly understand and use information. This constitutes a hard fracture.

E3: Minimum instruction following. The LLM must possess the minimum instruction following capability sufficient for engineering scaffolding—contracts, evaluations, checkpoint reviews, state handoffs—to take effect. T1 requires the system to restart from a snapshot—but if the LLM does not output the snapshot in the prescribed format, the subsequent executor receives an unparseable state-snapshot, and T1 cannot be realized in engineering. T2 requires self-verification by rubric—but if the LLM does not follow the verification instructions, the self-verification result is unreliable. Critically, the AC paradigm’s means of compensating for LLM deficiencies—clearer contracts, stricter evaluations, more frequent checkpoint reviews—themselves depend on the LLM following these compensation instructions. If the base model structurally lacks the minimum instruction following capability, the AC compensation layer undergoes **cascading failure**: the compensation instructions themselves are not followed. This constitutes a hard fracture; changing the base model is the only response.

4.1.3 Cognitive Limits of Human Drivers—Human Carrying Baseline

Once the engine satisfies the physical limits, checkpoints can be advanced. But whether the direction of advancement is correct depends on humans—and humans likewise have cognitive limits. This judgment inherits Simon’s^[3] classic account of bounded rationality: human cognitive resources are not unlimited; in complex decision-making, individuals do not seek a global optimum but search for a satisfying solution under conditions of incomplete information, limited computational capacity, and time pressure. The three human carrying baselines below do not rediscover this fact but translate it from general cognitive constraints into three actionable constraints within the long-horizon checkpoint context.

H1: Asymmetric verification bandwidth. The cognitive cost for a human to “read and understand the state” at a checkpoint must be strictly less than their currently available attention budget. Let $C_{\text{verify}}(c_i)$ be the human’s verification cognitive cost at checkpoint c_i , and $B(t_i)$ be the remaining attention budget upon reaching c_i ($B(t_i)$ is monotonically non-increasing with cumulative time t_i , i.e., $B(t_{i+1}) \leq B(t_i)$); then the following must hold:

$$C_{\text{verify}}(c_i) < B(t_i).$$

When this inequality is violated, humans produce **rubber-stamping**—formally going through the approval process while substantively skipping judgment. At this point, value closure becomes an empty ritual: completed in engineering form but not occurring in cognitive substance.

This constraint and the E1 constraint form a dual boundary for the checkpoint design space. E1 defines the lower bound ($|O_i| \geq L_{\min}$) and upper bound ($|O_i| \leq S$) of checkpoint granularity; H1 adds a temporal decay constraint—even if $S \geq L_{\min}$ holds, the human’s attention budget decays over time and may drop below the verification cost at some t_i . This means checkpoint spacing cannot consider only the LLM’s capability span but must also account for the human’s attention decay curve.

H2: External reality anchoring capacity. Humans must possess the ability to map the system’s internal consistency onto external reality value—to judge, among a set of logically self-consistent outputs, whether it still conforms to the needs and constraints of external reality. T2 ensures that the system can self-verify within the current context “has it been done according to contract,” but it cannot verify “does what has been done still have meaning.” When external drift or decision forks occur, the LLM as a closed information processing system is insensitive—it will continue perfectly executing an already obsolete objective. Humans must possess the cognitive leap ability to cross the gap of “system internal logic $\rightarrow$ external reality.” If humans are also laypersons in that domain, they can only see “no errors” (goal closure) but cannot see that “the direction has become wrong” (value closure failure). When absent, the response is not to add engineering compensation—compensation cannot help a layperson make judgments—but to narrow the scope of practice or change the judge.

H3: Decision braking and reset threshold. Upon receiving anomalous signals, deviation warnings, or value consultation requests from the machine, humans must possess the psychological and judgment threshold to overcome the system’s momentum of advancement, press the stop button, or reset the direction. T1 and Compensation 1 provide rollback capability at the mechanism level, but “whether to roll back” is a value decision—it can only be made by humans. The machine has advancement momentum: when the system issues a signal requiring human adjudication, if humans every time choose “default continue” due to fatigue, hesitation, or analysis paralysis, then the rollback mechanism is effectively null in engineering—the system has rollback capability but it is never triggered. Analysis paralysis is particularly fatal: if at a checkpoint humans face an open-ended question (“what do you think should be done?”), the cognitive energy consumed by the decision is extremely high, and humans tend to shelve it or default to approval. This explains why engineering protocols need to transform open-ended judgments into multiple-choice questions—lowering the cognitive cost of decision-making to ensure humans can brake within the threshold.

4.1.4 Irreplaceability of the Substrate Prerequisite

The substrate prerequisite is an admission condition, not an optimization item. For a system that does not possess minimum carrying capacity, adding more engineering patches cannot elevate it to a viable carrier system; patches can only reinforce a skeleton that already possesses capacity, not substitute for the capacity itself. Figure 4 presents the deductive structure of the three-layer substrate prerequisite and the blocking consequences of any missing layer.

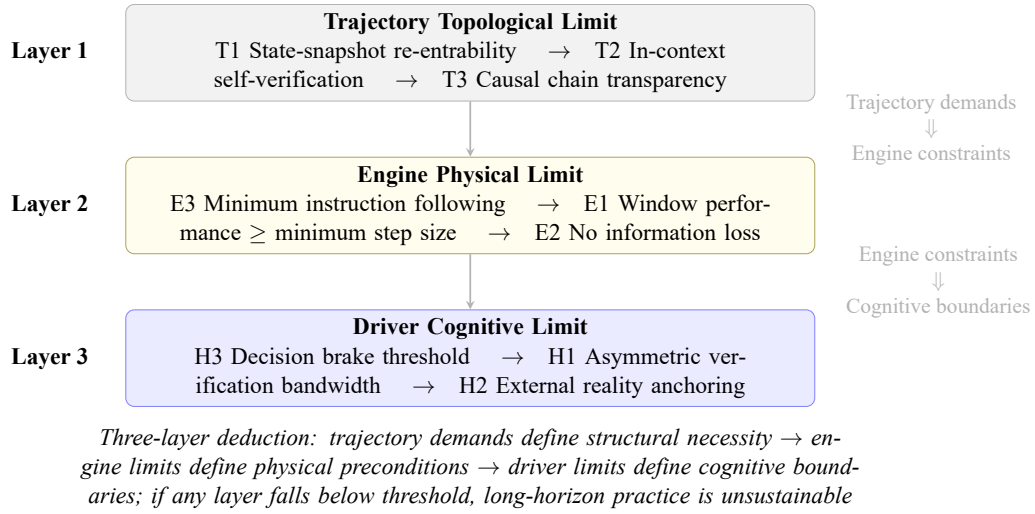


Figure 4: The three-layer deductive structure of substrate prerequisites. Layer 1, trajectory topological limit, defines the structural requirements that any long-horizon practice must satisfy (T1/T2/T3). Layer 2 derives the physical limits of the LLM as a computing node from trajectory demands (E1/E2/E3). Layer 3 derives the cognitive limits of the human as value arbiter from trajectory and engine constraints (H1/H2/H3). The three layers form a chain of deduction; none may be omitted.

4.2 Logical Necessary Conditions: Preventing Structural Fractures of Continuity

After the substrate prerequisite is established—i.e., the system possesses the minimum qualification to enter Long-Horizon Practice—six logical necessary conditions define how continuity is structurally maintained. Here, “necessary” does not mean “usually desirable” but rather that a clearly locatable structural fracture occurs when any condition is absent. Each condition is annotated with its fracture type: hard fracture means the practice physically or logically cannot continue; soft

fracture means advancement can superficially continue but the results are already unreliable.

4.2.1 Condition 1: Goal closure [hard fracture]

Root cause: Practice is a goal-driven closed-loop activity. Without a closure criterion defining “what counts as complete and what counts as incomplete,” the feedback loop cannot judge whether the current cycle is complete. Checkpoint decomposition and open-state marking both presuppose this—“unclosed” requires “closed” to be defined first in order to be judged. In plain terms, every step’s output must have a verifiable checkpoint.

Consequence of absence: The system can keep busy indefinitely yet be unable to distinguish whether it is advancing or drifting. Not knowing when something counts as complete means checkpoints cannot be defined.

Interlocking with upstream and downstream: Goal closure is the definitional prerequisite for checkpoint decomposition and open-state marking—without a closure criterion, “unclosed” is undefined, and “checkpoint” loses its anchor. Value closure acts upon it in a hierarchical governance relationship.

Guidance for engineering practice: The harness must mandatorily establish an actionable closure criterion (contract) before accepting a task; otherwise, entry into execution is prohibited.

4.2.2 Condition 2: Value closure [soft fracture]

Root cause: Goals are inherently local, and the correctness of a local purpose cannot be judged by the purpose itself. A higher-level value judgment is required—under the AC paradigm, this judgment right belongs to the human. Value closure is typically confirmed by human intervention at key checkpoints.

Consequence of absence: Advancement itself is not blocked—it can be completed, but it may be done in the wrong direction. The system may very efficiently head in the wrong direction.

Interlocking with upstream and downstream: Value closure acts hierarchically upon goal closure—“completed” does not equal “done correctly.” It is the existential prerequisite for Compensation 6 and Compensation 7: without value closure, machine-to-human explanation and consultation lose their functional anchor.

Guidance for engineering practice: Every key checkpoint must preset a human intervention node, and the trigger condition for that node must be based on explicit engineering signals rather than human self-awareness.

4.2.3 Condition 3: Checkpoint decomposition [hard fracture]

Root cause: If the overall objective is non-decomposable, the concept of “checkpoint” does not exist, and Long-Horizon Practice degenerates into single-turn gambling—either finish it all at once (which is not long horizon), or never advance at all. Decomposability does not mean “the human pre-splits everything”—decomposition can be performed by the agent, but decomposability must objectively exist.

Consequence of absence: Long-Horizon Practice degenerates into a single-turn task—unable to advance segmentally, unable to verify locally, unable to interrupt and resume.

Interlocking with upstream and downstream: Once checkpoint decomposition is established, ordering constraints and state transfer acquire their clear operational objects. The decomposition granularity must precede the point where resource dissipation causes execution capability to fall below the executable threshold—this design constraint points to the dual boundary of E1 and H1 in the substrate prerequisite.

Guidance for engineering practice: Before accepting a task, the decomposability of the overall objective must be verified—if non-decomposable, the harness should refuse acceptance rather than attempt to brute-force it in a single-turn manner.

4.2.4 Condition 4: Ordering constraint [soft fracture]

Root cause: Decomposability answers “can it be split”; ordering constraints answer “how to organize after splitting.” If one only knows that a task can be decomposed but does not know the temporal dependency relationships among subtasks, parallel division of labor is unreliable—two subtasks may wait for each other or overwrite each other.

Consequence of absence: Advancement can continue, but will involve constant rework and spinning idle. When a strict partial order is entailed (a predecessor’s output is the logical input of a successor checkpoint), the soft fracture can escalate to a deadlock approximating a hard fracture—the successor checkpoint physically cannot start.

Interlocking with upstream and downstream: Ordering constraints presuppose checkpoint decomposition—without checkpoints, dependency relationships have no operational objects. For a parallel harness, they are the existential prerequisite for serial-parallel strategy.

Guidance for engineering practice: The harness should detect and annotate partial order relationships; for strictly partial-order dependencies, implement a blocking gate; for non-strict dependencies, merely annotate the risk.

4.2.5 Condition 5: State transfer [hard fracture]

Root cause: The closure result of checkpoint N must be acquirable by checkpoint $N + 1$. But “state is acquirable” (e.g., Git records file changes) is not equivalent to “state is handoff-ready”—the new executor must understand why the file is in this state, what remains unfinished, and where to continue from next. Hence, state transfer inherently requires semantic annotation: where we have reached, why we have reached here, what is closed, what is unclosed, and where to pick up from next.

Consequence of absence: The initial conditions of the successor checkpoint are missing, and the practice physically cannot be resumed. Every handoff is like reopening the topic from scratch; previous judgment bases, constraints, and unresolved items evaporate at the state-snapshot.

Interlocking with upstream and downstream: State transfer presupposes checkpoint decomposition and deeply interlocks with open-state marking—the transferred state snapshot must contain unclosed information. Compensation 6 is its engineering supplementation in the human-to-machine direction.

Guidance for engineering practice: The harness must mandatorily execute a state snapshot at each checkpoint closure, and the snapshot must contain semantic annotation rather than merely file content. Without state transfer, the next step is prohibited from starting.

4.2.6 Condition 6: Open-state marking [soft fracture]

Root cause: The system must be able to distinguish three states—“closed,” “unclosed,” and “currently indeterminable”—and must prohibit recording the latter two as “closed.” The core is not about demanding honesty from the executor—that is unreliable—but about requiring the system structure to independently produce unclosed signals. Test failures, type errors, and contract verification failures are such signals; “I think it should be fine” is not.

Consequence of absence: Advancement can continue—“everything looks complete” can go on indefinitely—but erroneous intermediate states are passed backward as completed premises, and the entire dependency chain is built on pseudo-closures.

Interlocking with upstream and downstream: Open-state marking presupposes goal closure—“unclosed” requires “closed” to be defined first in order to be judged. It is the logical prerequisite for Compensation 3 (independent evaluation)—independent evaluation is the engineering-enforced implementation of open-state marking.

Guidance for engineering practice: The harness must mandatorily output a three-valued state annotation at each checkpoint node and prohibit replacing in-

dependent evidence with the executor’s self-report.

The above six conditions are not arranged as a flat list. Goal closure provides the definitional prerequisite for checkpoint decomposition and open-state marking; once checkpoint decomposition is established, ordering constraints and state transfer acquire their clear objects; value closure, as a higher-level judgment, runs through them, continuing to constrain “whether it was done correctly” beyond “whether it was finished.”

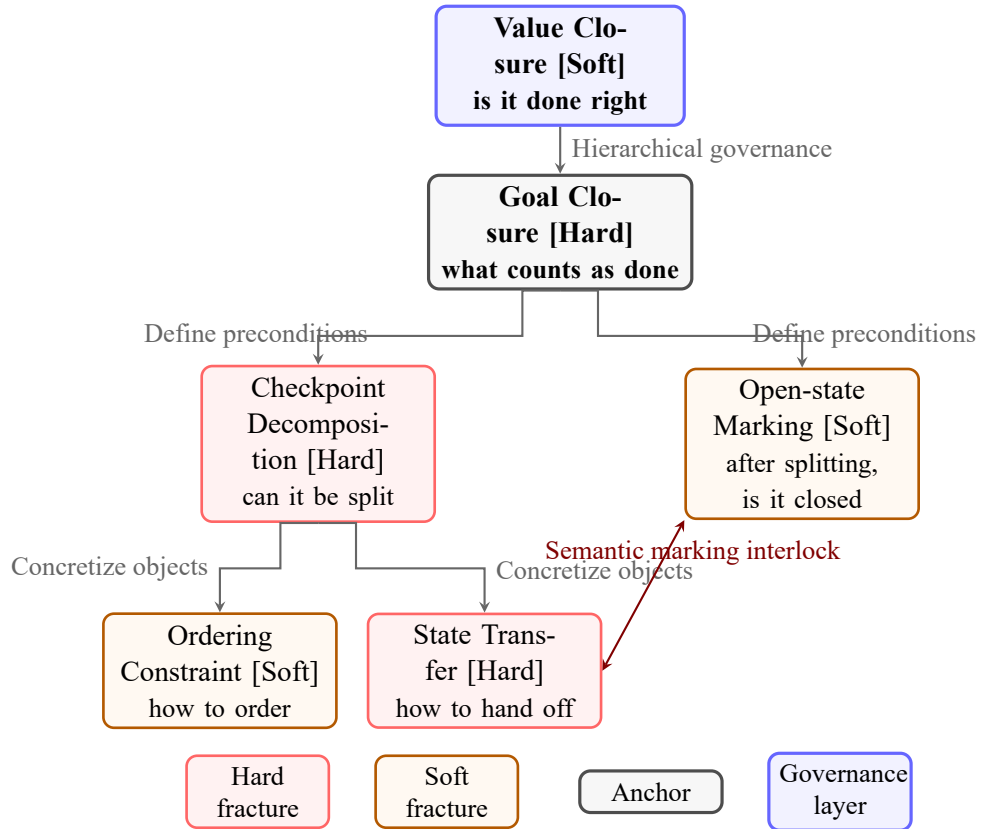


Figure 5: Structured dependency relations among the six logical necessary conditions. Goal closure provides the definitional anchor for checkpoint decomposition and open-state marking. Once checkpoint decomposition is established, ordering constraint and state transfer acquire concrete operational objects. Value closure spans above through a hierarchical governance relation, constraining “is it done right.” State transfer and open-state marking interlock deeply through semantic marking, ensuring information does not evaporate at handoff points. The hard fracture and soft fracture labels indicate the structural consequences of missing each condition.

These six conditions are not abstract classifications. In the controlled spec-

imen of Appendix A, each corresponds to a concrete object on the real task trajectory: goal closure corresponds to the non-downgradable GATE table for that problem; checkpoint decomposition corresponds to the multi-stage advancement from combinatorial decomposition to algebraic order reduction; state transfer corresponds to the bidirectional synchronization between spec and current-note and the maintenance of the obsolete index; open-state marking corresponds to the three-valued state annotation “17/21 passed, 4 cases to be conquered”; value closure corresponds to the “prohibited retry routes” list as an externalized boundary for human adjudication. These objects are not best practices collated after the fact but structures that the task was forced to grow in order to survive across fracture zones.

4.3 Engineering Compensation Conditions: Placing the Skeleton into the Real World

Even when both the substrate prerequisite and the logical layer are established, real practice will still be disrupted by various high-probability noises: execution will drift off course, errors will spill over, evaluations will be replaced by the executor’s self-report, and humans will lose awareness or miss intervention opportunities at key nodes. What the engineering compensation conditions address are precisely these parts that are “logically doable but easily broken in engineering.” They are not substitutes for the substrate prerequisite, nor substitutes for the logical necessary conditions, but rather the armor layer that makes the lower-bound system actually operable in a real environment.

4.3.1 Compensation 1: Recovery and rollback

When closure judgments are erroneous, execution drifts off course, or the environment changes, the system must be able to return to a known valid state. A single erroneous closure judgment should not permanently contaminate the entire practice chain. However, the rollback decision right must belong to the human—a rollback is itself an irreversible decision, and the rationality of this decision can only be judged by the value closure layer. It primarily reinforces state transfer and open-state marking: when errors are discovered, the system needs the ability to isolate the error within a recoverable boundary, rather than having no choice but to press on.

4.3.2 Compensation 2: Isolation and boundary

The system must limit the error propagation radius, preventing a single local mistake from contaminating the entire practice unit. Even if checkpoints have been

decomposed, without boundary constraints, a single erroneous modification may contaminate multiple successor checkpoints together. Isolation does not replace checkpoint decomposition but limits the propagation radius of a single mistake, keeping errors localized, diagnosable, and within a rollback-able range.

4.3.3 Compensation 3: Independent evaluation

Acceptance must not depend on the executor’s self-report. It must be borne by tests, checkers, type systems, manual acceptance, or other independent evidence chains. It is the engineering-enforced implementation of open-state marking—ensuring that open-state signals do not depend on the executor’s self-report but come from a mechanism that can be independently falsified.

4.3.4 Compensation 4: Instruction following compensation

“Information arrival” does not equal “information executed as intended.” When an executor (especially an LLM) has a natural tendency to override external intent with its own preferences, merely writing down rules and contracts is insufficient—mechanisms such as contract explicitation, checkpoint review, and deviation detection are needed to continuously calibrate the gap between “instruction received” and “instruction followed.” It forms a symmetry with substrate prerequisite E3: E3 addresses “whether the base model has the capability to follow”; Compensation 4 addresses “whether a capable base model deviates during execution.”

4.3.5 Compensation 5: Capability gap escalation

When the executor itself does not possess the capability required to complete a certain step—for instance, the LLM has complete knowledge gaps for a language, framework, or paradigm—the system must allow explicit handover, reassignment, or introduction of external capability, rather than encouraging the executor to fill the gap with hallucination. Distinction from open-state marking: open-state marking handles “knowing that something is unfinished” (known unknowns); capability gaps handle “not knowing that something needs to be known”—the executor does not even understand what the task means. The effectiveness of the escalation mechanism depends on one premise: the existence within the system of a capability superior who can take over. If the knowledge gap is global (the LLM does not know, the human does not understand, and no external tool is available), the task exceeds the current system’s carrying upper bound—AC can expose the gap and prevent pretending to advance, but cannot conjure knowledge out of thin air.

4.3.6 Compensation 6: Proactive machine-to-human explanation

The premise for the human to make a value judgment at a checkpoint is understanding the current state. But LLMs naturally tend to omit decision rationales, skip steps, or use reasoning paths that are hard for humans to trace—this creates a structural information asymmetry: at the checkpoint, the human faces a state that “has already become like this” without knowing “why it became like this.” The system must require the machine, at key nodes requiring human intervention, to proactively provide a minimum explanation package: the completion status of the current checkpoint, key decisions and their rationales, unclosed items, questions requiring judgment, and risk warnings. It is the engineering supplementation of state transfer in the human-to-machine direction: Condition 5 guarantees LLM $\rightarrow$ LLM state transfer; Compensation 6 guarantees LLM $\rightarrow$ human state transfer. Neither can be omitted.

4.3.7 Compensation 7: Proactive machine-to-human inquiry and alignment

LLM executors have a systematic tendency: when encountering value ambiguity, boundary conditions, or insufficient self-judgment, they tend to “fill the gap with their own preferences” rather than “expose the gap for human decision.” This causes value drift that can accumulate across multiple checkpoints without being detected. The system must require the machine to proactively pause and consult the human in the following situations: encountering non-technical value trade-offs, reaching the boundary of its own judgment capability, detecting silent drift between the execution path and the contract, or when the current decision may produce an irreversible path-locking effect on subsequent checkpoints. Compensation 7 together with Compensation 6 constitutes the engineering protocol layer of human-machine interaction—Compensation 6 addresses “is the human informed when present”; Compensation 7 addresses “does the human need to be present.” Together they ensure that although value closure is definitionally the human’s responsibility, there are also engineering mechanisms ensuring the human can actually fulfill this responsibility.

4.4 Fracture Type to Condition System Mapping

Placing the four information cutoff sources of §3 alongside the condition system yields a clearer conclusion: different fracture types do not compress the same set of conditions; they each push the system toward failure along different paths. Figure 6 presents the four main fracture paths.

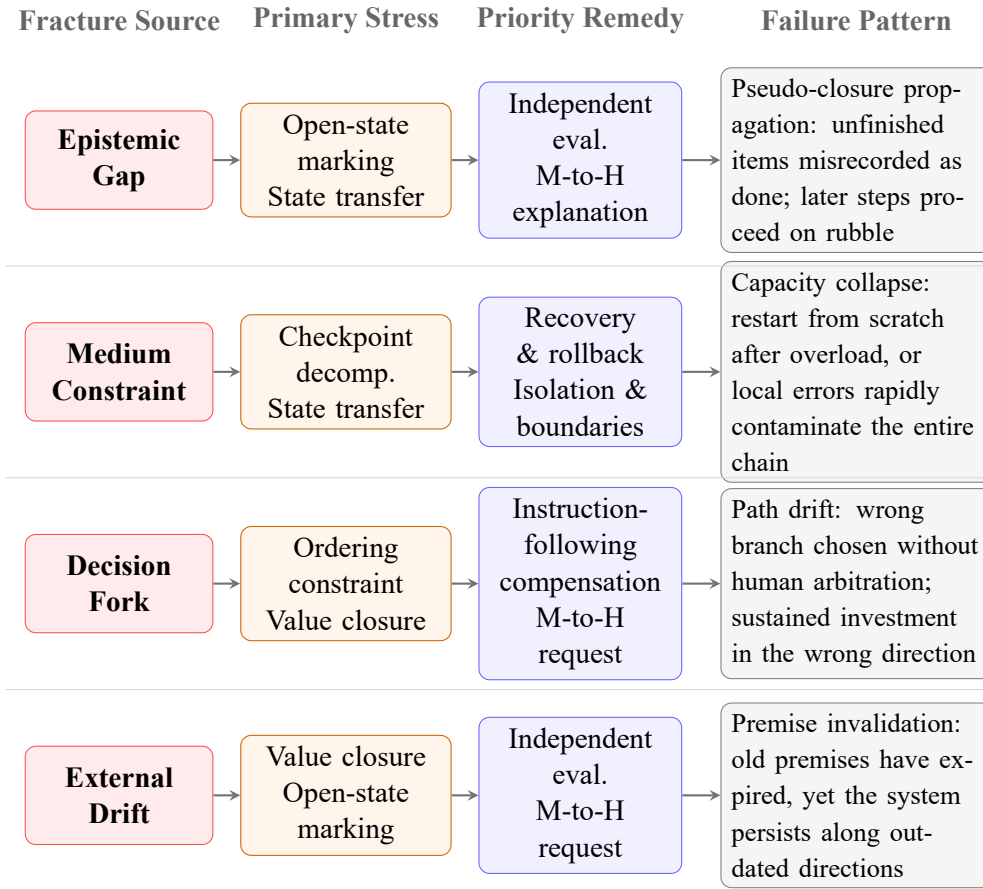


Figure 6: The four fracture types each penetrate the condition system along distinct paths. Epistemic gap and external drift share open-state marking but invade from different directions—the former prematurely closes when the truth value has not yet been generated, while the latter refuses to reopen when the truth value has already expired. Medium constraint and decision fork respectively attack the granularity constraints and ordering constraints of checkpoints. The four paths do not overlap; there is no universal patch that can handle all fractures uniformly.

This mapping has two layers of meaning. First, different fracture sources correspond to different weak surfaces, so “adding a few more patches to the system” cannot uniformly solve the problem. Second, engineering compensation conditions can only reinforce the logical layer they correspond to but cannot leap across levels to replace it. Independent evaluation can help expose pseudo-closures but cannot replace the logical requirement of open-state marking; machine-to-human consultation can ensure that value nodes are not silently bypassed but cannot replace value judgment itself; recovery and rollback can contain deviations within known boundaries but cannot patch a system lacking carrying capacity into a vi-

able one.

5 Cross-Domain Generality: Structure, Not Subject Matter, at Work

If one accepts the premise of §2—that the agent is itself the perception-action loop, and that practice and intelligence are two inseparable phases of the same loop—then the structural requirements of Long-Horizon Practice are not contingent constraints tied to coding tools or LLM architectures but necessary constraints tied to the loop structure itself. The task is to prove this point: first distinguish structural variables from subject-matter variables, then use fractal isomorphism to show how the same constraint equation is instantiated across the two dimensions of construction and execution, and finally present a non-coding case to directly demonstrate the applicability of the analytical framework.

5.1 Structural Variables and Subject-Matter Variables

Definition 2.4 uses the checkpoint sequence $C = (c_1, c_2, \dots, c_n)$, the current information set I_i , the key input K_{i+1} , and the sequential dependency relation for its judgment—these are **structural variables**: they describe how information unfolds along the time axis, independent of the specific discipline of the task content. In contrast, “writing code,” “doing mathematics,” and “designing buildings” are **subject-matter variables**: they describe the content domain of the information.

The determination of Long-Horizon Practice depends only on structural variables: whenever there exists some i such that $K_{i+1} \notin I_i$, and that dependency cannot be folded by precomputation, the system has entered the Long-Horizon Practice regime. This determination does not care whether K_{i+1} is “the signature of the next function” or “the client’s real evaluation of the proposal.” Likewise, the four sources in §3—epistemic gaps, medium constraints, decision forks, and external drift—describe four structural causes of $K_{i+1} \notin I_i$, not four industry phenomena. The three-layer lower bound in §4—substrate prerequisite, logical necessary conditions, engineering compensation conditions—describes the minimum structural qualification required to cross these fracture zones, not the minimum competence of some type of engineer.

Thus, claiming that this framework possesses cross-domain generality is a testable structural proposition: **in any domain where there exists a non-precomputable sequential dependency chain, the information structure will encounter the same structural problems at the same locations.** Conversely, even if a task is massive in scale and extremely time-consuming, if the required information is

largely complete at the starting point and all parts can be decomposed in parallel, it is not Long-Horizon Practice—the information structure is different, so the structural problems are different.

5.2 Fractal Isomorphism: Construction and Execution Share the Same Constraint Equation

The point where structural generality is most easily misunderstood concerns functional positioning. Some accept that “designing architecture and planning systems” is Long-Horizon Practice, yet believe that “debugging algorithms and locating errors” is mere technical execution, unrelated to structural issues; others conversely acknowledge that stepwise optimization of complex algorithms requires cross-checkpoint advancement, yet see system architecture design as a one-off intellectual judgment, as if “doing things” has sequential dependencies while “thinking about things” can be accomplished in a single mental step. Both miss the same point: in Long-Horizon Practice, construction and execution are not two different types of activity but the same information structure recurring under different functional positions.

This judgment can be directly tested by substituting the formal variables. Take a typical trajectory from the construction side and one from the execution side of the same software project:

- **Construction:** checkpoint sequence $C^{(\text{constr})} = (c_1, c_2, c_3) = (\text{requirements}, \text{architecture}, \text{delivery})$. Before c_1 is completed, the key input $K_2^{(\text{constr})}$ required by c_2 (module boundaries, interface constraints, technology selection premises) has not yet entered $I_1^{(\text{constr})}$ —they depend on the artifacts of requirements analysis to be determined.
- **Execution:** checkpoint sequence $C^{(\text{exec})} = (c_1, c_2, c_3) = (\text{locate}, \text{modify}, \text{verify})$. Before c_1 is completed, the key input $K_2^{(\text{exec})}$ required by c_2 (the root cause rather than the symptom, the specific scope of modification) has not yet entered $I_1^{(\text{exec})}$ —they depend on the actual output of debugging and the analysis results of the stack trace.

Juxtaposing the two trajectories, the overlap in structural relations becomes immediately apparent:

$$\begin{aligned} \text{Construction : } & K_2^{(\text{constr})} \notin \text{Fold}(I_1^{(\text{constr})}), \quad K_3^{(\text{constr})} \notin \text{Fold}(I_2^{(\text{constr})}) \\ \text{Execution : } & K_2^{(\text{exec})} \notin \text{Fold}(I_1^{(\text{exec})}), \quad K_3^{(\text{exec})} \notin \text{Fold}(I_2^{(\text{exec})}) \end{aligned}$$

The **content** carried by I_i and K_{i+1} on the two sides is entirely different—one side carries requirements documents and architecture diagrams, the other carries

error messages and line numbers. But the **constraint relation** $K_{i+1} \notin \text{Fold}(I_i)$ coincides pairwise across the two dimensions—like two systems of equations with identical coefficients but different variable names, they are the same object in algebraic terms. This is the precise meaning of fractal isomorphism: contents differ, constraint equations are isomorphic.

This conclusion is not an analogy but the same information topology instantiated twice across the two dimensions of construction and execution. It has two direct consequences. First, the three-layer lower bound of §4 does not presuppose functional positioning: the requirements of checkpoint decomposition, state transfer, open-state marking, and value closure will be raised separately on the construction and execution levels, differing only in their implementation forms—on the construction level they may be milestone reviews and version baselines, on the execution level test cases and diff reviews. Second, a long-horizon system is not one where “the top layer has plans and the bottom layer relies on improvisation”; if the execution layer entirely lacks a structure of the same kind, the continuity of the construction layer will sooner or later be subverted by local fractures—if $K_{i+1}^{(\text{exec})} \notin \text{Fold}(I_i^{(\text{exec})})$ is ignored, the resulting errors will propagate upward along the dependency chain and eventually contaminate $K_j^{(\text{constr})}$.

5.3 A Minimal Cross-Domain Anchor: Architectural Proposal Review

Architectural proposal review is a sufficiently short, sufficiently coding-distant example that can directly demonstrate the applicability of the analytical framework.

After the design team submits Proposal A, the client then says “not enough meeting rooms”; the team modifies it into Proposal B according to feedback, and the client then says “too many meeting rooms, not enough office space”; Proposal C again triggers new feedback that “the budget is exceeded.” Every step in this process follows the same type of information structure:

- **Epistemic gap:** Before seeing Proposal A, the client does not truly possess the judgment of “whether the number of meeting rooms is reasonable.” K_{i+1} (the client’s real need for spatial allocation) has not yet been generated before the proposal is submitted—this is a typical case of not yet generated, and it is unrelated to architectural expertise.
- **Decision fork:** Upon receiving the feedback, the team chooses the modification direction of “reduce meeting rooms, increase office area”—but this decision itself will affect what new feedback the client produces later. The true value of K_{i+2} depends on the intermediate decision made by the team at c_i , rather than being fixed at the starting point.

- **External drift:** The “budget exceeded” constraint was only made explicit after Proposal C was submitted—it was not that the client deliberately concealed it, but that in the first two rounds, the client’s attention was occupied by the meeting room problem, and the budget constraint had not yet entered the effective discussion space. The task boundary is continuously rewritten during the interaction process.

Three fracture types naturally appear in a single non-coding case, and the four-source classification of §3 directly covers them without any cross-domain adaptation. What does not appear in this case is the medium constraint—but if the review takes place between transnational teams, time differences, language translation, and asynchronous communication would precisely introduce the medium constraint. In other words, the four sources do not all manifest simultaneously in non-coding domains, but the conditions under which they appear are exactly the same as in the coding domain: so long as $K_{i+1} \notin I_i$, at least one explanation will be found among the four structural causes.

5.4 Closing

The structural lower bound of Long-Horizon Practice describes an information structure, not a body of trade experience. It takes structural variables as its objects and $K_{i+1} \notin I_i$ as its criterion; thus in any domain where the same information structure appears, the fracture sources and carrying requirements will reappear in the manner of fractal isomorphism. The surface content will change: code, blueprints, diagnostic reports differ from one another; the constraint equations will not change: the information cutoff between checkpoints and its conduction along the time axis will always follow the same structural relation.

It is precisely because the closed-loop structure does not change with the “material” of the agent (human, LLM, or human-machine composite) that the condition system established in §4 does not presuppose a specific subject matter or a specific agent type. This is a structural derivation: so long as the relations among C , I_i , K_{i+1} , and $K_{i+1} \notin I_i$ exist, the four types of fracture causes and the six logical necessary conditions for carrying continuity will come into effect, independent of the specific content of C , I_i , and K_{i+1} . Section 6 demonstrates the predictable ways in which real systems fail when these conditions are absent.

6 When Single-Step Closure Encounters Long-Horizon Structure: Three Types of Structural Failure

§2 defined the information structure of Long-Horizon Practice, §3 explained four sources of fracture, §4 presented the three-layer structural lower bound required for crossing fractures, and §5 proved that this framework does not depend on a specific subject matter. After a theoretical framework has been shown to be logically self-consistent, one more thing needs to be demonstrated: how it can reinterpret empirical phenomena already familiar to the reader. Otherwise, the reader may concede its logical coherence but will not find it useful.

The preceding sections (§2 through §5) have already completed the proof. The definitional lens of §2 and §3 is now used to examine three types of typical failures that recur in current long-horizon agent engineering, allowing the reader to see clearly: those problems that appear unrelated to each other and are often attributed to “the model not being strong enough” or “the engineering not being refined enough” are structurally the same thing—a single-step closure system attempting, at a checkpoint where information is not yet sealed, to forcibly close the next step using materials already at hand.

6.1 Failure I: Compensating with More Context When Information Has Not Yet Been Generated

When a long-horizon task fails, the most common engineering response is to enlarge the context window—injecting more conversation history, more code files, more retrieval results into the current inference. The underlying assumption is: the failure arises because “relevant information was not seen by the model,” so as long as the window is large enough, the problem can be solved.

§3 distinguished four sources of information being currently unavailable. Enlarging the context window, in essence, handles only one of them: the medium constraint (information already exists but the carrier is limited). But for epistemic gaps (information not yet generated) and external drift (information belongs to future states), no matter how large the window, it cannot fold truth values that have not yet collapsed into it. The error message after a piece of code runs, the client’s real feedback after seeing the proposal, the measurement result of the next experiment—these truth values do not yet exist at the current checkpoint. Injecting one million tokens of conversation history into the window merely brings more of the “past” into view, without pre-generating the “future.”

What the context window compensates for is retrieval breadth, not temporal generation. When the truth value has not yet appeared in the world, window size is irrelevant to correctness.

6.2 Failure II: Substituting Memory for State, Self-Report for Verification

Most agent systems treat multi-turn conversation history as the sole carrier of task state: when resuming after an interruption, they re-feed the previous conversation records to the model, expecting it to “pick up.” Meanwhile, as soon as the agent outputs “completed” or “I have fixed the issue,” the system defaults that checkpoint to closed, rarely introducing independent external verification.

§2 defined checkpoints and the current information set: what checkpoint closure requires is a structured state-snapshot of “what is closed, what is unclosed, and where to pick up from next,” not a temporal memory record of “what happened before.” Conversation history belongs to the latter—it records the sequence of events but does not explicitly annotate which premises have been verified, which remain to be confirmed, or the dependency chain state of the current state-snapshot. Conversation history is memory, not state. Substituting memory for state is like asking the next shift’s worker to read all of the predecessor’s diaries instead of giving them an engineering blueprint.

§4’s open-state marking (Condition 6) requires the system to independently produce closed/unclosed signals rather than depending on the executor’s self-report. When the system substitutes self-report for verification, it is performing the same kind of operation as Failure I: passing information identities that have not yet been independently confirmed onward as closed premises. In a long-horizon dependency chain, self-reported closure without independent verification is, in essence, the forced global injection of the executor’s local judgment.

6.3 Failure III: Substituting Local Convenience for Global Judgment at Nodes Requiring Value Adjudication

When agents encounter obstacles—compilation errors, test failures, interface conflicts—the most common strategy is to find, within the local scope, the fastest path to clear the current obstacle. Delete a validation logic segment, bypass a type constraint, substitute an approximate value for an exact calculation. Each step looks “reasonable” locally, but after accumulating several steps, the system has already deviated from the original objective.

§3’s decision fork points out: when the information identity needed by the next step depends on branch choices not yet made, the system structurally cannot determine in advance what to collect and what to optimize. When the executor, at such fork points, does not wait for adjudication but autonomously chooses the path of least local cost, it is in effect treating a fork that requires an external value judgment to close as an optimization problem that can be closed within the execution layer through technical metrics. §4’s value closure (Condition 2) and

machine-to-human consultation (Compensation 7) are precisely designed for such nodes—they require that the system not allow the execution layer itself to decide between “doing it fast” and “doing it right.”

The local reasonableness of each step, accumulated over several steps, may be equivalent to a systematic deviation of the global direction—and yet at every step, the system believes it has done the right thing. Local convenience can quickly clear the current obstacle but cannot substitute for directional judgment.

6.4 Closing: A Shared Structural Root Cause

On the surface, the three failure types cover different problem domains—context management, state handoff, path selection—but under the framework of §2 and §3, they share the same structural root cause: a single-step closure system attempting, at a checkpoint where information is not yet sealed, to forcibly close the next step using the materials it already has. Whether it is compensating for ungenerated truth values with more context, compensating for structured state with conversation memory, or compensating for value judgment with local convenience, in essence they are all doing the same thing—“pre-closing” at c_i an information identity that properly belongs to c_{i+1} to be determined.

This explains why local improvements such as enlarging windows, enhancing retrieval, and optimizing prompts cannot eradicate long-horizon failures: they all optimize within the same closed information set without changing the structural fact that “the current information set is not closed with respect to the next step.”

In this sense, mechanisms such as rollback and isolation, structured handoff, independent verification, and proactive consultation that have grown out of the engineering iteration of systems like Claude Code^[4] are not the design preferences of a particular company but natural evidence of independent engineering teams being repeatedly forced toward the same set of carrying design patterns under real constraints. The evolution of SWE-bench from single-turn correctness toward real repositories, real tests, and real intermediate feedback^[5-6], as well as the sustained emphasis on mixed-initiative design in human-machine collaboration research^[7-8], each converge from different angles on the same judgment: long-horizon capability is not the sum of single-step capabilities but another type of structural qualification.

This is not external endorsement but corroboration—it shows that the three-layer lower bound in §4 is not an aesthetic preference collated after the fact but a common threshold that independent systems repeatedly encounter in long-term survival. From these structural insights, corollaries about capability dimensions can be drawn.

7 Conclusions and Corollaries

§2 through §6 have established a single argument chain: the formal definition of Long-Horizon Practice $\rightarrow$ four sources of key information unavailability $\rightarrow$ the three-layer structural lower bound required for crossing fractures $\rightarrow$ a structural proof of cross-domain generality $\rightarrow$ the explanatory power over three types of structural failure. This section does not repeat the derivation but instead draws from this argument chain corollaries about capability dimensions, recovers the contributions of this paper, and clarifies the boundaries of the current framework.

7.1 Corollary: The Three-Stage Non-Implication of Capability Dimensions

Long-Horizon Practice cannot be reduced to a mere sequence of single-step closures—§2 established this structurally, and §6 provided corroboration through three types of failure. From this follows a corollary about evaluation dimensions: there exists no direct implication relation between single-step capability and long-horizon capability. What is at issue here is not a difference of degree but a difference of dimension—they measure structural qualification in different dimensions.

Proposition 7.1 (Strong single-step capability $\nRightarrow$ single-checkpoint closure). *A model that performs excellently in single-turn or short-context settings does not imply that it can stably close a real engineering checkpoint.*

Structural rationale: §4 has already shown that a real checkpoint is not a single-turn problem with a longer prompt—it requires the system to probe the boundary of what it knows and does not know (Condition 5) and to distinguish which constraints cannot be crossed at the execution level on its own. Local solution correctness does not imply that the checkpoint can be closed under global constraints.

Proposition 7.2 (Single-checkpoint closure $\nRightarrow$ sustained cross-checkpoint progress). *Even if the system can locally close and deliver a single checkpoint, it does not imply that it can sustain progress in a task chain containing multiple checkpoints.*

Structural rationale: The sequential dependency defined in §2 dictates: after crossing a checkpoint, errors enter subsequent steps through the handoff boundary, and this is no longer a matter of answering one question wrong. State transfer (Condition 1), reversible rollback (Condition 3), and closed-loop verification (Condition 6) are the structural prerequisites for accumulating local correctness into global correctness, and they cannot be automatically satisfied by local closure—Failure II of §6 has already demonstrated this.

Proposition 7.3 (Sustained cross-checkpoint progress $\nRightarrow$ high single-turn benchmark scores). *A system possessing a long-horizon carrying structure may not achieve scores matching its survival capability in evaluations that emphasize instantaneous hit rate and single-turn output quality.*

Structural rationale: Compensation 7 (machine-to-human consultation) of §4 and Failure III of §6 have jointly shown: long-horizon systems proactively introduce verification, rollback, and consultation mechanisms. These behaviors are counted as costs on the single-turn evaluation scale, yet they are precisely the structural price the system pays to maintain continuity under information-incomplete conditions—they are admission conditions for sustaining long-horizon continuity, not optional redundancies.

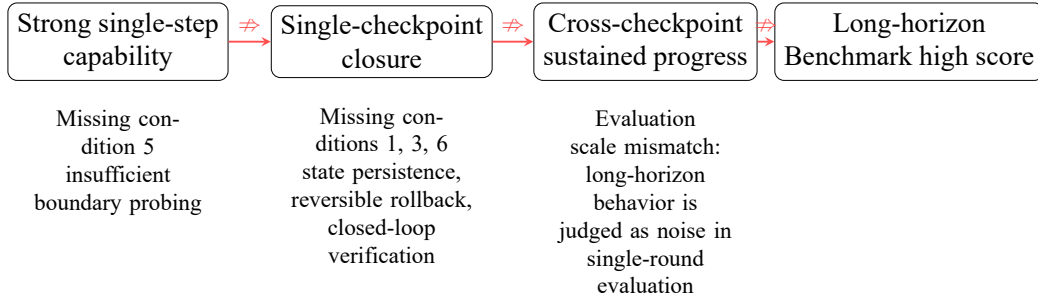


Figure 7: A three-link non-implication chain. Strong single-step capability does not entail single-checkpoint closure (missing boundary probing). Single-checkpoint closure does not entail cross-checkpoint sustained progress (missing state persistence, reversible rollback, and closed-loop verification). Cross-checkpoint sustained progress does not entail a high long-horizon benchmark score (evaluation scale mismatch).

The three-stage non-implication points to the same judgment: evaluating long-horizon capability requires a long-horizon ruler. The current evaluation system centered on single-turn hit rates and short-context output quality measures instantaneous performance within local closures; yet the core difficulty of Long-Horizon Practice—the stepwise generation of information along the time axis and the maintenance of cross-checkpoint continuity—lies outside the measurement range of these rulers. This is not a matter of evaluations not being good enough but of measuring different dimensions.

7.2 Contributions Recovered by This Paper

First, the minimum judgment skeleton of Long-Horizon Practice. Through the formal definition of checkpoints, current information sets, key inputs, and sequential

dependencies, this paper tightens “long horizon” from a workload description into a structural criterion: whenever the truth value required by the next step cannot be pre-closed within the current information set, the task has already entered the long-horizon regime. The engineering significance of this criterion lies in this: it transforms “does this task actually need a carrying structure” from a post-hoc empirical judgment into a formal check executable at c_i .

Second, an exhaustive classification of the sources of key input unavailability. Epistemic gaps, medium constraints, decision forks, and external drift constitute a complete partition of the logical space of “why $K_{i+1} \notin I_i$.” The closing segment of §3 unifies the four sources into a single principle: feedforward computation cannot substitute for state transition. The role of this classification is to provide an exhaustive fracture diagnostic matrix for the subsequent condition system.

Third, the three-layer structural lower bound of Long-Horizon Practice. The substrate prerequisite (track topology $\rightarrow$ engine physics $\rightarrow$ driver cognition) answers “whether there is carrying capacity”; the six logical necessary conditions answer “how continuity is structurally guaranteed against fracture”; the seven engineering compensation conditions answer “how this skeleton survives in real-world noise.” Crucially, the openness of the engineering compensation layer is not a theoretical defect—what is open is the engineering form of compensation implementation, not the structural criteria of carrying capacity and the logical skeleton themselves.

Fourth, the capability-dimensional corollary naturally drawn from the above argument chain. §5 proved that this framework does not depend on a specific subject matter; §6 proved that three types of engineering failure share the same structural root cause; the corollaries are condensed into three citable propositions: strong single-step capability $\Rightarrow$ single-checkpoint closure $\Rightarrow$ sustained cross-checkpoint progress $\Rightarrow$ high single-turn scores. The three-stage non-implication is not a denial of existing evaluations but a boundary delineation of the measurement range of the current evaluation system.

7.3 Limitations and Boundaries

The first limitation concerns logical status. What this paper provides is a structural lower bound—a set of necessary conditions. When these conditions are satisfied, the system possesses carrying capacity; when they are not, Long-Horizon Practice will structurally fracture at some state-snapshot. As for how to achieve better performance after satisfying the lower bound, that falls within the domain of capability upper bounds and strategy optimization, outside the main argument scope of this paper. A further clarification is needed: the formal symbols introduced in this paper— C , I_i , K_{i+1} , S , $L_{\min}$, etc.—currently reside at the topological structure description level. Physically quantifiable definitions that can be directly substituted

into engineering measurement (for example, a numerical determination method for $L_{\min}$ in a specific task) have not yet been provided. This is not a defect of the theory but a deliberately left interface: the entire derivation from §2 to §4 shows that these variables precisely constitute the mandatory measurement axes for evaluating long-horizon capability—the task design, checkpoint granularity determination, and closure criteria of next-generation long-horizon benchmarks all essentially need to be built around them. The seven engineering compensation conditions, within this framework, are “structural approximations forced out by engineering practice,” not already-proven sufficient conditions—their openness is precisely an honest expression of this limitation.

The second limitation concerns the verification boundary. The empirical support of this paper comes mainly from post-hoc dissection (such as the controlled specimen of Appendix A) and the natural convergence of independent engineering practice (such as the corroboration in §6), belonging to qualitative structural pathology diagnosis. It should be noted that the v6 engineering implementation of the AC paradigm targeting the Trae environment has been completed and has passed stress testing on real long-horizon tasks—this constitutes prospective engineering verification, but that portion falls outside the theoretical argument scope of this paper and will be presented separately as an independent technical report. Large-sample, cross-task-family quantitative causal analysis remains for future work; what the present framework provides is the structural foundation for Long-Horizon Practice, not yet extended into a dynamic model capable of computing fracture probabilities.

The third limitation concerns engineering friction. The seven engineering compensation conditions proposed in this paper—independent evaluation, proactive machine-to-human explanation and consultation, state snapshots and semantic annotation, recovery and rollback, etc.—provide the necessary noise buffer for long-horizon continuity but simultaneously introduce significant execution friction: every checkpoint demands additional verification steps, handoff records, and audit nodes. In short tasks or scenarios where information is already largely closed, this “step-by-step entrenched” structure incurs unnecessary overhead and may even suppress the advantage of large models in rapidly producing results based on fuzzy intuition—it trades instantaneous execution efficiency for long-horizon stability. Therefore, this framework should not be generalized as the default standard for all LLM interactions; its value is activated if and only if the condition $K_{i+1} \notin I_i$ holds; imposing this mechanism in scenarios that do not require long-horizon carrying is a net negative.

The fourth limitation concerns scope of applicability. What this paper discusses are tasks in which there exists a non-precomputable sequential dependency chain. When the required information is already complete at the starting point, sub-problem boundaries can be given all at once, and the entire path can be folded in

advance, single-step closure tools are already sufficient. In such cases, the role of this framework is boundary clarification of “do not misjudge as long horizon”—it provides a distinguishing criterion, not an additional burden imposed on scenarios that do not require long-horizon carrying.

7.4 Open Questions

First, how to realize the lower-bound conditions as a runtime substrate. The real difficulty lies in turning substrate prerequisite detection, state transfer, rollback, independent verification, and consultation boundaries into runtime primitives of the joint system—so that they no longer depend on the executor’s on-the-spot self-awareness. The independent engineering practice corroborated in §6 shows that this is not fantasy, but concrete implementation schemes for different architectures (single-agent, multi-agent, human-machine composite) still require separate design.

Second, how to establish an evaluation framework oriented toward long-horizon capability. The argument chain of this paper has already indicated the direction: if single-turn hit rates and instantaneous output quality are insufficient to represent long-horizon qualification, evaluation tasks must explicitly include sequential dependency chains, open-state tracking, interruption recovery, and boundary probing. More fundamentally, the formal variables established by this paper— C (checkpoint sequence), I_i (current information set), K_{i+1} (key input), S (effective execution span), $L_{\min}$ (minimum verifiable step length)—directly provide a structural blueprint for such evaluations: a long-horizon-oriented benchmark should have its skeleton designed around “whether information cutoff between checkpoints is explicitly modeled, when key inputs are generated or blocked, whether state transfer is measured, and whether open-state marking is independently verified,” rather than continuing a precision competition on single-turn hit rates.

Third, nearly every combination of a different agent architecture with a different large model base requires independent testing and design. The structural criteria are cross-domain and cross-model—but engineering implementation is not. Different base models differ markedly in checkpoint closure behavior, information attrition patterns, and instruction decay rates; the optimal engineering form of the same set of compensation conditions under different combinations awaits case-by-case verification.

Thus, what this paper has accomplished is to draw an a priori boundary: in Long-Horizon Practice, no matter how strong the single-step capability, it cannot substitute for the carrying structure itself.

A Controlled Dissection: A Structural Specimen of Long-Horizon Practice

This appendix does not discuss the mathematical interest or competitive significance of the bracket-string counting problem below. We treat it as a controlled dissection specimen: the input and output are mechanically decidable, the closure criteria are strict (all test cases passed + time/space constraints + no degradation), and the exploration trajectory has complete anchor-point documentation. What concerns us is not how the solver ultimately cracked the problem, but rather how to reconstruct the process along the actual record: which key inputs could only emerge during the process, which locally correct steps could not naturally accumulate into global closure, and what structure the task ultimately depended on to cross the successive fractures.

A.1 Problem Statement

The following is the complete original description of the problem. Bracket-string DP is a well-known combinatorial optimization problem, but in this appendix we are not concerned with its competitive background—we treat it only as a controlled specimen whose information structure can be precisely reconstructed.

Time limit: 10 s **Memory limit:** 1024 MB

Problem Description

Alice and Bob like to study bracket strings.

A string $s = s_1 s_2 \cdots s_{2n}$ of length $2n$, consisting only of the characters $($ and $)$, is called **valid** if and only if:

- for any prefix, the number of left brackets is no fewer than the number of right brackets;
- the whole string contains exactly n left brackets and n right brackets.

For a valid bracket string s of length $2n$, define $\text{alice}(s)$ as the number of subsequences of s equal to the string $"()"$. In other words, $\text{alice}(s)$ equals the number of pairs (i, j) satisfying $1 \leq i < j \leq 2n$ with $s_i = ($ and $s_j =)$.

For a given integer a , Alice defines $\text{SA}(s) = a^{\text{alice}(s)}$.

Bob devised a different counting method for the same string s . Initially set $l = 0$ and $\text{bob}(s) = 0$. While $l < n$, repeat the following process:

1. Let x be the maximum positive integer such that in the $2 \times l + x$ -th prefix, the number of left brackets $($ is exactly $l + x$;
2. Update $l \leftarrow l + x$;
3. Update $\text{bob}(s) \leftarrow \text{bob}(s) + (n - l)$ (note the execution order: update l first).

It can be proved that this process always proceeds to completion, eventually reaching $l = n$, and at every step $x \geq 1$ holds.

For a given constant b , Bob defines $\text{SB}(s) = b^{\text{bob}(s)}$.

Given an integer n , for each integer i ($1 \leq i \leq n$), let $\mathcal{D}_i$ be the set of all valid bracket strings of length $2i$. Compute:

$$F_i = \sum_{s \in \mathcal{D}_i} \text{SA}(s) \times \text{SB}(s) = \sum_{s \in \mathcal{D}_i} a^{\text{alice}(s)} \times b^{\text{bob}(s)}.$$

Since the answer may be very large, output $F_i \bmod 998\,244\,353$.

Input

One line containing three integers n, a, b ($1 \leq n \leq 5000$; $1 \leq a, b < 998\,244\,353$).

Output

One line containing n integers, where the i -th integer is $F_i \bmod 998\,244\,353$.

Examples

Input #1

3 1 1

Output #1

1 2 5

Input #2

4 2 3

Output #2

2 40 4544 2624512

Correct solution trajectory. The final solution to this problem went through the following key stages:

1. After exhausting multiple candidate routes (A~K), full-scale numerical verification at $n = 8$ confirmed that the problem is equivalent to the (q, t) -Catalan numbers known in combinatorics^[9-10];
2. The Garsia-Haglund recurrence^[11] was used to transform the problem into a $O(N^3/6)$ dynamic programming formulation;

3. Under the judge’s time limit pressure, the complexity was reduced to $O(N^2)$ through q-factorial separation and NTT (Number Theoretic Transform) convolution;
4. After handling boundary conditions such as the existence of modular inverses, all test cases were passed.

Every step of this solution depended on new information generated after the previous step was executed—the equivalence could not be derived by “thinking,” the complexity could not be predicted by “analysis,” and the boundary conditions could not be known at the outset. It is, by nature, a structural specimen of long-horizon practice.

A.2 Specimen Selection: Why This Problem is Suitable for Dissection

First, the problem is a closed formal system. Given $n \leq 5000$, modulus 998244353, time limit 10 s, memory limit 1 GB, compute the sum of $a^{alice(s)}b^{bob(s)}$ over all valid bracket strings of length $2i$ —the input and output are completely formalized, with no natural language ambiguity. This rules out the “the real world is too messy” excuse: no failure can be attributed to misunderstanding of requirements or specification drift.

Second, the closure criteria are fully mechanically decidable. The judge system returns a single AC/WA/TLE verdict, with no fuzzy intermediate states such as “correct direction” or “roughly correct.” All 21 test cases passed, time ≤ 10 s, memory ≤ 1 GB, single-file implementation, no non-standard degradation strategies—these five conditions together constitute an automatically executable closure criterion. This rules out the “evaluation criteria are vague” excuse.

Third, the recurrence scale of this problem directly triggers the medium constraint defined in §3. The combinatorial explosion of Catalan numbers and the $O(N^2)$ DP ($N = 5000$) produce a state space that far exceeds any human working memory or LLM context window—the equivalence conclusions, falsified directions, and boundary condition criteria that each step depends on can no longer be stably loaded into the current information set at the start of the next step. This means: it is not that “good records enable replay,” but that “without records, progress is impossible.” The structural records generated during the task—the verified-true / verified-false table in the spec file, the deprecation index, the list of prohibited retry routes, the judge feedback history, the GATE closure table—are not an engineering convenience, but a structural compensation for medium constraint. This rules out the alternative explanation of “the model just isn’t strong enough or has a bad memory”: the long-horizon nature comes from objective constraints on information structure, not from the executor’s cognitive limitations.

Fourth, external drift is virtually absent in this problem. The problem constraints, test data, and judge behavior remained stable throughout the task. Precisely because of this, if it still exhibits significant long-horizon character, it means the long-horizon nature is not a byproduct of real-world noise, but a result of the internal information structure of the task—the fact that key inputs are progressively generated along the time axis holds even in the absence of external world changes.

Data availability note. The complete original research materials of this specimen—including all checkpoint execution trajectories, falsification history, anchor files (spec.md, current-note.md, etc.), and judge feedback records—have been open-sourced at https://github.com/jopsammy/task1_raw_repo, for readers to independently verify against the analysis in this paper.

A.3 Checkpoint Reconstruction: State Snapshots of Cognitive Progress

The following reconstructs the task trajectory according to the structural framework of long-horizon practice. Each entry in the table corresponds to one information-generation stage. Each stage answers three questions: what information is currently available, what key input is still missing for the next step, and why that key input is not yet in the current information set.

Table 1: Checkpoint sequence (C1–C3): the point-by-point emergence of information growth and epistemic gaps

ID	Currently Available Information	Missing Key Input	Why Unavailable
C1	Problem constraints ($N \leq 5000$, 10s, 1GB, MOD= 998244353), alice is tractable along the standard Catalan decomposition, the per-prefix batch-scan definition of bob	Which decomposition paradigm can simultaneously accommodate the tree structure of alice and the x -sequence structure of bob	This truth value cannot be determined by thinking alone—it must be fixed by actually writing the DP, running exhaustive verification, and comparing against real wrapping data. Before these actions are executed, the world has not yet generated this information
C2	Multiple candidate routes (A~K) have been explored and recorded, some have been partially verified	Whether these routes achieve global closure under real wrapping	Wrapping is equivalent to the full height function, whose behavior cannot be losslessly predicted by any low-dimensional DP state. In other words, there is no method cheaper than “execute verification scripts route by route” to obtain this truth value—it must be executed route by route
C3	Many routes have been falsified and blocked	Whether the problem is equivalent to a known combinatorial object $((q, t)$ -Catalan)	The equivalence cannot be “seen”—it requires mapping alice/bob to the area/bounce statistics of Dyck paths, then performing full-scale numerical verification at $n = 8$ (a total of 1430 Dyck words). Before the full-scale verification is complete, the equivalence is only a hypothesis, not a fact

Table 2: Checkpoint sequence (C4–C7, continued)

ID	Currently Available Information	Missing Key Input	Why Unavailable
C4	Equivalence to (q, t) -Catalan confirmed, Garsia-Haglund recurrence available ^[11] , complexity $O(N^3/6)$	Whether $O(N^3/6)$ can pass the judge within 10s	The judge’s microarchitecture (CPU model, cache hierarchy, compiler optimization strategy) is unknown; a local 8.89s does not imply the judge will also pass. This truth value does not exist before submitting to the judge—the verdict must actually be triggered
C5	v11 judge verdict 17/21, 4 cases timed out; 208 billion dot products confirmed as the algebraic lower bound	Whether there exists a path to reduce the order at the algebraic level without changing the loop structure: q-factorial separation + NTT convolution	Whether q-factorial separation is valid must be determined after completing formal derivation and $n \leq 8$ full-scale verification. Before the derivation is complete, this information does not yet exist—it is not “known but inconvenient to compute”
C6	v14 NTT path passes full-scale $n \leq 8$, $N = 5000$ runs in 7.66s	Whether modular inverses exist for all possible a	A zero divisor in the q-factorial appears only when the multiplicative order of some a is $\leq N + 2$. Before the pre-detection is complete, this truth value cannot be predicted—it must be fixed by computation before the world can settle it
C7	v15 judge verdict 19/21, 2 cases WA	Whether the error originates from algorithm design or implementation details (integer overflow, modular arithmetic boundaries)	The error category can only be determined after pinpointing the specific test case and analyzing the multiplicative order of the corresponding a . Before the localization is complete, the category of the error is itself an undetermined piece of information

Table 1 and Table 2 list the seven information-generation stages where epis-

temic gaps are most salient. Each stage contains numerous sub-level snapshots (the elimination of falsified routes one by one, the sequential response to judge feedback, etc.), but the structural feature—“key inputs do not exist as determinate facts before the triggering action is executed”—recurs consistently across all seven stages. This is precisely the pure form of epistemic gap described in §3 of the main text.

A.4 Fracture Localization: Concrete Manifestations of the Four Sources in a Closed System

The following maps the four source categories from §3 of the main text onto the actual trajectory of the specimen. We do not repeat definitions here; we only provide structural slices.

Epistemic gap (the strongest source in the specimen). Three decisive truth values did not exist before the triggering actions were executed: (1) whether wrapping can be determined by local parameters—this truth value did not exist before the exhaustive comparison at $n = 4$ (14 Dyck words) and $n = 8$ (1430 Dyck words) was executed; (2) whether the Garsia-Haglund recurrence is equivalent to the original problem—this truth value did not exist before the $n \leq 8$ full-scale numerical verification was completed; (3) whether the judge can run $O(N^3/6)$ within 10s—this truth value did not exist before submitting to the judge. These three truth values are not “hidden facts waiting to be discovered,” but “facts that must be generated by the world through executing specific actions.” This is precisely the defining characteristic of the epistemic gap: the truth values themselves are progressively generated along the time axis, rather than progressively discovered along the spatial axis.

Decision fork (the second-strongest source in the specimen). The successive forks from route A through L constitute a typical decision-fork chain: after the tree-decomposition-based wrapping localization approach (routes A~D) was falsified, the effort turned to the x -sequence-based batch merging approach (routes E~H), then to the $L(i)$ sequence representation (routes I~K). Each eliminated route consumed real exploration and verification costs (exhaustive script runtime, formal derivation time), and the elimination conclusions themselves had to be stably preserved as permanent boundaries for the subsequent search space—i.e., the “prohibited retry routes” list. The paths not selected were not “not attempted,” but “attempted and falsified,” and their falsification conclusions serve as common premises for all subsequent steps.

Medium constraint (weaker but genuinely present in the specimen). Across 102 hours of multi-day progress, the entire failure history, deprecated conclusions, exception conditions, current best version, and unclosed-item list could not be sta-

bly compressed into a single-round context or a single session. The externalized anchor-point system of `spec.md` and `current-note.md`—maintaining cross-day state via the three-segment handoff structure (engineering process / handoff status / final result)—is precisely the engineering response to medium constraint. The medium constraint is not the most dramatic fracture source here, but it is the precondition for continuity to be maintained: without externalized anchors, cross-day progress would degrade to restarting from scratch each time, completely disconnected from the previous day’s accumulated reasoning.

External drift (virtually absent in the specimen—this is a strength). This problem is a closed formal system; the external world state remained essentially unchanged during the task. The task constraints, test data, and judge behavior all remained stable. The specimen’s coverage of “external drift” is weak, but this is precisely its value as a controlled specimen: after excluding external drift, the task still exhibits significant long-horizon character across the entire C1→C7 trajectory. This shows that the long-horizon nature does not arise from “the world changes,” but from the more fundamental structural fact that “key information is progressively generated along the time axis.”

A.5 Continuity Repair: How the Carrying Structure Emerges Naturally

The following presents the structural objects that the specimen actually depended on during its progress, and maps them to the three-layer lower bound of §4 in the main text. The core claim here is: these objects were not institutions designed in advance by the solver, but structures that were forced to grow under fracture pressure.

Table 3: Structural objects → main-text conditions → mechanisms of forced emergence

Structural Objects Present in the Task	Corresponding Main-Text Condition	How They Were Forced to Emerge in the Specimen
GATE table (21/21 AC + time + memory + single-file + non-degraded)	Goal closure	Not a design preference—without it, 17/21 passed would be merely a vague sense of progress rather than an operable closure criterion; the executor could not determine “when is it considered done”
“Prohibited retry routes” list	Value closure	Not a management requirement—without it, subsequent executors would repeatedly fall into previously falsified old routes (the blocked items among routes A~K), with each retry consuming real exploration cost
Multi-stage progression from combinatorial decomposition to algebraic order reduction	Checkpoint decomposition	Not pre-planned—stage boundaries formed gradually only after each old paradigm (tree decomposition, x -sequence, $L(i)$ representation) was falsified. The boundaries of checkpoint division are determined by fracture positions, not by a roadmap drawn in advance
Bidirectional synchronization between spec.md and current-note.md	State transfer	Not a documentation norm—without it, upon cross-day recovery, the previous day’s judgment basis and blocked-route list would evaporate at the state-snapshot, and the new session would not know which routes had been eliminated, which version was the current best, or which counterexamples had been discovered
The three-state marking of verified-true / verified-false / deprecated	Open-state marking	Not a classification fetish—without it, overturned conclusions (such as the early hypothesis that “ $(A)B$ decomposition holds for bob”) would be mistakenly reused as premises in subsequent steps, causing derivation waste built on a wrong foundation
Deprecation index (permanent traces of overturned conclusions)	Ordering constraint + state transfer	Not an archival fetish—without it, the knowledge boundary after successive forks cannot be maintained. Abandoned routes need to be “remembered for why they were abandoned,” otherwise the system would oscillate back and forth between old pits and new pits

The six-row mapping in Table 3 collectively reveals a pattern: every struc-

tural object was not designed because “it ought to be done this way,” but was demanded by the task itself because “if it weren’t done this way, the task would destabilize on the fracture zone.” These objects are the load-bearing skeleton of long-horizon practice, not embellishments of best practice.

A.6 Why the Bare Model Consistently Fails

The following argument does not provide a conclusion about “whether the model is strong enough.” Instead, it lets the structural reasons emerge on their own along the chain of evidence from the preceding four subsections.

The bare model does not lack local reasoning capability. A large number of local steps in the specimen—the recognition of Catalan structures, the equivalence judgment of (q, t) -Catalan, the algebraic derivation of q -factorial separation, the design of NTT convolution—are all genuine mathematical insights at the single-step reasoning level. In the early stages $C1 \rightarrow C3$, single-step reasoning capability was sufficient to complete exhaustive verification, equivalence confirmation, and initial recurrence design.

What the bare model lacks are the continuity objects that make local correctness accumulable. The following six categories of objects are the necessary carriers for transforming local correctness into global closure—they are not stably held by default within a single-round conversation:

1. Replayable falsification history: knowing which routes have been eliminated and why;
2. Stable prohibited-retry boundaries: falsified routes will not be retried in new sessions;
3. A single source of truth (spec.md): all executors share the same list of “currently known to be true”;
4. Resumable state-snapshot after interruption (the three-segment handoff of current-note.md): upon cross-day recovery, the previous day’s judgment basis is not lost;
5. Closure evidence independent of the executor’s self-report: judge feedback, full-scale verification script output, not “I think it’s correct”;
6. The ability to rewrite the entire roadmap when new counterexamples appear: the path upgrade from v14 to v15 (changing q -factorial from a factor structure to NTT convolution), which requires the executor to return to the more upstream hypothesis layer and re-judge, rather than merely appending a patch.

These six categories of objects are the load-bearing skeleton of long-horizon practice. Their common feature is that none of them lies within the input-output space of single-step reasoning; they all belong to the meta-structure necessary for “maintaining continuity across checkpoints.”

The reason for failure is structural, not intellectual. The bare model’s failure on the specimen is not because some particular step was miscalculated, but because the above six categories of objects are not stably held in the default single-round conversation. Going further—even if we assume the existence of an infinitely intelligent single-step reasoner, it could not, within a single session, foresee the globality of wrapping (C2), the equivalence of Garsia-Haglund (C3), the judge’s microarchitecture (C4), or the zero divisors of bad a (C6)—because these truth values simply do not exist in the world before they are actually executed and verified. The problem does not lie inside the solver, but in the gap between the solver and the continuity objects. As long as this gap is not separately identified and compensated for, the intensity of local reasoning will not automatically translate into cross-checkpoint closure.

A.7 Specimen Boundaries: What It Covers and What It Does Not

Strong coverage. Epistemic gap (throughout C1→C7), medium constraint (the combinatorial explosion of Catalan numbers and the $O(N^2)$ DP state space far exceeding the upper bound of the context window; the anchor-point system is a structural compensation for this), decision fork (routes $A \sim L$), goal closure (GATE table), checkpoint decomposition (from combinatorial decomposition to algebraic order reduction), state transfer (bidirectional spec/note synchronization), open-state marking (three-state marking). These are the structural dimensions most fully demonstrated by the specimen.

Moderate coverage. Value closure (the “prohibited retry routes” list as an externalized boundary of human arbitration). The specimen provides real but limited evidence on this dimension.

Weak coverage. External drift (a closed problem; the task environment remained essentially unchanged), and most of the engineering compensation conditions (real-world noise—personnel turnover, requirement changes, toolchain upgrades—is substantially reduced in a closed system).

These “weak coverages” are not defects of the specimen. Precisely because external drift is excluded, the specimen proves that the root of long-horizon character lies not in environmental noise, but in the more fundamental structural fact that information is generated along the time axis. The weak coverage of engineering compensation conditions, in turn, illustrates that long-horizon practice in a closed world is primarily constrained by the logical layer (the 6 necessary conditions of §4 in the main text), while long-horizon practice in the real world additionally requires the noise-buffering of the compensation layer (the 7 compensation conditions). The two are not contradictory, but a reasonable difference between

the specimen and reality—by reducing real-world noise, the specimen renders the structural necessity of the logical layer visible.

B Acknowledgments

We thank the Qianjiang New Town Central Business Industry Community, Shangcheng District, Hangzhou, for its indispensable support in survey coordination, foundational stress-testing samples, and real-world business use cases; the Hangzhou Synergy Referral Platform Community (SRO) for providing substantive use case support; the members of the Comet Project for their technical assistance; and numerous members of the AI-Zion community for thought-provoking discussions and valuable suggestions.

We are grateful to DeepSeek for providing Deepseek-v4-pro as the base model, on which the 102-hour stress run of the appendix specimen was conducted under the Anchor-Contract Human-Machine Collaborative Engineering Paradigm (AC Paradigm V6). We also acknowledge TRAE for critical IDE environment technical support and Cherry Studio for essential technical support.

References

- [1] LIU F, LV B, LIU Y. From Observer to Agent: On the Unification of Physics and Intelligence Science[Z/OL]. 2024. <https://www.preprints.org/manuscript/202410.0479/v1>.
- [2] LIU N F, LIN K, HEWITT J, et al. Lost in the Middle: How Language Models Use Long Contexts[J/OL]. Transactions of the Association for Computational Linguistics, 2023, 11: 157-173. <https://arxiv.org/abs/2307.03172>. DOI: 10.1162/tacl_a_00638.
- [3] SIMON H A. A Behavioral Model of Rational Choice[J/OL]. Quarterly Journal of Economics, 1955, 69(1): 99-118. DOI: 10.2307/1884852.
- [4] Anthropic. Best Practices for Claude Code[EB/OL]. 2025 [2026-05-21]. <https://www.anthropic.com/engineering/claude-code-best-practices>.
- [5] JIMENEZ C E, YANG J, WETTIG A, et al. SWE-bench: Can Language Models Resolve Real-world Github Issues?[C/OL]//The Twelfth International Conference on Learning Representations. 2024. <https://openreview.net/forum?id=VTF8yNQM66>.
- [6] CHOWDHURY N, AUNG J, SHERN C J, et al. Introducing SWE-bench Verified[EB/OL]. 2024 [2026-05-21]. <https://openai.com/index/introducing-swe-bench-verified/>.

- [7] AMERSHI S, WELD D, VORVOREANU M, et al. Guidelines for Human-AI Interaction[C/OL]//CHI 2019. 2019. <https://www.microsoft.com/en-us/research/publication/guidelines-for-human-ai-interaction/>.
- [8] HORVITZ E. Uncertainty, Action, and Interaction: In Pursuit of Mixed-Initiative Computing[J/OL]. Intelligent Systems, 1999: 17-20. <https://www.microsoft.com/en-us/research/publication/uncertainty-action-interaction-pursuit-mixed-initiative-computing/>.
- [9] GARSIA A M, HAIMAN M. A Remarkable q, t -Catalan Sequence and q -Lagrange Inversion[J/OL]. Journal of Algebraic Combinatorics, 1996, 5(3): 191-244. DOI: [10.1023/A:1022476211638](https://doi.org/10.1023/A:1022476211638).
- [10] HAGLUND J. The q, t -Catalan Numbers and the Space of Diagonal Harmonics: vol. 41[M]. American Mathematical Society, 2008.
- [11] GARSIA A M, HAGLUND J. A Proof of the q, t -Catalan Positivity Conjecture[J/OL]. Discrete Mathematics, 2002, 256(3): 677-717. DOI: [10.1016/S0012-365X\(02\)00343-6](https://doi.org/10.1016/S0012-365X(02)00343-6).